Python under load: a measurement study of JIT, runtime, native code and build-level optimisations

T. S. Ivankov
ITMO University

Scientific advisor: I. V. Struchkov, Cand. Sc. (Tech.)
ITMO University

Abstract

Deciding how to speed up a Python program means choosing among a dozen techniques whose published results were not produced under comparable conditions — different machines, different input sizes, different baselines — and almost never state what the technique costs on the code it does not accelerate. This paper measures the whole menu on one machine, under one protocol, with one interpreter provenance. Six kernels — three compute-bound, three branchy and allocation-heavy — are implemented in pure Python, Cython, Numba, Codon (ahead of time and through its JIT), C (through `ctypes`) and Rust, with the three compute-bound kernels additionally in NumPy and a pybind11 binding; CPython 3.10–3.14 and the free-threaded build are compiled here from release tarballs with one compiler and one `configure` line; seven build configurations of 3.14.6 come from one source tree, including PGO with full LTO and the experimental tier-2 JIT; the PyPy 7.3.20 release binary, and Meta's Cinder fork with its CinderX extensions built from source on the same host, are measured against the same baseline. All timing is done by `pyperf`, so every number is a median over values from many independent processes, except where the text says otherwise; where a conclusion turns on two configurations being the same, it carries a significance test rather than a pair of rounded medians.

Six results are worth stating up front. On compute-bound loops the choice among the compiled routes is worth far less than the decision to compile at all — they agree to within 4.4% on a scalar reduction and differ by 21% on a mandelbrot grid and by 2.8× on a naive matrix product, in both directions and with no consistent winner, against the 31–128× that compiling at all is worth — while what separates a 32× result from a 77× one on the same kernel is the floating-point contract, not the tool. That contract is only partly a compiler flag: disassembling every flag variant shows `-O2` and `-O3` emitting identical machine code on two of the three kernels and indistinguishable times on the third, while `-march=native` changes the code of all three and the run time in both directions — 1.25× faster on the mandelbrot grid, 1.80× *slower* on the matrix product. What `-ffast-math` does to the reduction is break a serial dependency chain, and the source can break it without the flag. What limits a compiler on irregular code is allocation rather than branching: C and Rust lead Cython and Numba by 3.7–5.3× exactly where small objects are allocated, by at most 17% on a branch-heavy byte scanner, and not at all on a graph traversal. Interpreter progress is real but unevenly distributed: 3.14 runs attribute and method operations 2.0–3.0× faster than 3.10, 3.11 is where most of that arrived, and 3.12 gives part of it back by running slower than 3.11 on 14 of our 16 operations. Free-threading returns 5.6–6.1× on eight threads for a 1.05–1.14× single-thread cost paid by the build rather than by the lock, and code whose hot loop is already native scales as well or better without it (7.2×). Composition is profile-directed rather than additive: on a mixed pipeline the one change that matched the stage profile gave 1.35×, while stacking all four techniques gave 1.26× at best and 0.73× under the GIL.

Two rows concern the ground the rest stands on. Building the interpreter with PGO alone is worth 1.02×; the gain is in the combination, since PGO with *full* LTO is 1.10× and adding `-march=native` to it 1.12×, and the one operation that loses under all of them is attribute writes. CPython's own experimental JIT is 1.04× overall, improving 10 of our 16 operations and slowing 6 of them. On the alternative-runtime side, CinderX's JIT runs our six untyped kernels at 0.33–0.79× stock once it is allowed to compile after the interpreter has specialised — compiled before first call, the same kernels run at 0.85–1.53×, so the sign of that result is a property of the measurement — and its Static Python adds a further 11.4× on a typed kernel (7.5× for a process that draws the slow code layout), though those primitive types are worth having only because something compiles them: interpreted, even after we completed the specialising path that shipped unfinished, they are 5.8× slower than the boxed code they replace. The largest factor available without editing the source is PyPy, which runs our unmodified pure-Python kernels 3.2–42.8× faster but whose one `ctypes` call that hands a two-megabyte buffer across the boundary costs 5.3× more than CPython's while the calls that pass only integers are within 7%: "switch runtime" and "move the hot loop to C" exclude each other only where the boundary carries bulk data. All code, all `pyperf` result files and the figure and table generators are in the accompanying artifact; every figure and table here is regenerated from those files by the generator commands of §14.

1 Introduction

A team with a Python program that is too slow has a dozen remedies to choose between, and they differ less in what they promise than in what they demand. Some ask for nothing but a different executable: PyPy, or since 3.13 the experimental JIT inside CPython itself, or a newer release, or the same release built with different flags (PGO, LTO, architecture-specific tuning). Some ask for annotations and a build step: Cython, Numba, Codon. Some ask for a second language behind an FFI: C, C++, Rust. Some ask for a different runtime and the maintenance that comes with it: Meta's Cinder and its CinderX extensions [3], with a JIT, a static typing mode, a parallel collector and lazy imports. Choosing badly among these is expensive in engineering time, and the published evidence does not narrow the choice.

These options are well known. What is missing is a way to place them side by side: each is normally demonstrated on the kernel that suits it, on an unnamed machine, against an unnamed baseline, with no statement of what was held constant and no account of what the technique costs when it is not the one being recommended. A team deciding between them is left comparing numbers that were not produced under comparable conditions, and the questions that decide the outcome in practice — how large the input has to be before a compiler pays for itself, what a technique costs on the code it does not accelerate, whether two of them compose — are usually not answered at all.

This paper measures that menu under the conditions set out in §3. What we set out to produce is a table indexed by *what the workload does* rather than by tool, whose entries say *what helped, by how much, and what it cost*: compute-bound work, branchy and pointer-heavy work, object-oriented and allocation-heavy work, mixed pipelines, and the system level below all of them. Table 8 is that table, and every row of it is produced by the artifact accompanying this paper. Concretely, the contributions are:

1. A measurement setup (§3) in which all timing is done by `pyperf` and every claim that two configurations are the same carries a significance test, together with a correctness gate: an implementation is timed only after it reproduces the reference result at full problem size.
2. Eight measured questions (§4–§11), each answered with code, a benchmark and a figure, and each closed with the practical rule it supports.
3. A cross-technology comparison over six kernels — three compute-bound, three branchy/allocating — in seven implementations, which separates the effect of *compilation* from the effect of *data representation* and puts the boundary between the two classes at allocation rather than at branching.
4. A machine-code account of what compiler flags actually change (§4.1): the disassembly of every flag variant is hashed, which shows that most of the flags usually recommended emit byte-identical code for these kernels, and that the one effect worth several times the run time belongs to a dependency chain rather than to a flag — demonstrated by reproducing it in the source at unchanged flags.
5. A measurement-based maturity report on Meta's Cinder fork and CinderX, built from source with identical configure flags against CPython 3.14.6, in *both* of CinderX's build configurations, covering the JIT, Static Python, lazy imports, the parallel garbage collector and lightweight frames (§7). Measuring the adaptive configuration at all first required completing it: the specialising path for static opcodes was carried across to 3.14 without the handlers it specialises into, so Static Python could not execute a single specialisation. The missing half is supplied in the artifact and verified against CinderX's own 3910-test static suite.
6. An artifact (§14) that regenerates every figure and table here from the `pyperf` result files.

2 Related work and the questions we measure

Comparative performance work on Python implementations is usually one of three kinds. (i) Benchmark-suite reports for a single implementation: the CPython `pyperformance` [2] results that accompany each release, and the PyPy speed pages [14]. These are rigorous and longitudinal, but they compare an implementation with itself over time, not one acceleration strategy with another. (ii) Documentation for an individual accelerator — Numba, Cython, Codon — which demonstrates the kernels the tool is good at, by design. (iii) Practitioner articles that time one kernel two or three ways; useful as hypotheses, but typically stating no platform, no input size and no protocol, and no cost.

What is comparatively rare, and what this paper does, is a study that (a) holds the workload fixed while varying the acceleration technology, (b) holds the technology fixed while varying the runtime, (c) reports the same statistics for both, and (d) quantifies the price of each technique alongside its gain. The measurement tooling is standard where standard tooling exists: all timing is `pyperf` [1], the benchmark runner used for CPython's own performance work, so the protocol below is its protocol rather than one of our own.

Table 1: The measurement host. Every field is read back from an artefact the run produced: the machine description written on the host itself, and `pyperf`'s per-run metadata.

property	value
CPU	Intel Core Ultra 7 255H
cores	6 performance + 10 efficiency cores
memory	30.8 GB
operating system	Ubuntu 24.04.4 LTS, Linux 7.0.0-28-generic (x86_64)
frequency scaling	governor performance, turbo off
address randomisation	Full randomization
benchmark affinity	CPUs 0-5 (the performance class)

The eight questions follow the workload classes of §1. Each section states its question, describes the experiment, and closes with what the numbers support.

Q1 How much do JIT compilation and native code actually buy on compute-bound kernels, and is the win the language, the compiler, or the freedom the compiler is given (Q1b)? What does the compilation itself cost before it pays (Q1c)?

Q2 Branchy and pointer-heavy code resists compilation. Which property of such code is responsible, and how much does moving it out of Python return?

Q3 What did the interpreter's own evolution (3.10 → 3.14: specialising bytecode, immortal objects, allocator work) deliver for object-heavy code?

Q4 Cinder and CinderX offer a JIT, a static typing mode, a parallel collector, lightweight frames and lazy imports. Which of them pay off, and at what cost?

Q5 Free-threaded scaling, and the single-thread price of the build that allows it.

Q6 The interpreter as a C program: what PGO, LTO and architecture flags are worth on one source tree, and what its own tier-2 JIT is worth (Q6b).

Q7 Composition on a realistic mixed pipeline.

Q8 PyPy on the same kernels.

3 Methodology

The machine. One, described in Table 1, and every number in this paper comes from it — including the Cinder and CinderX measurements of §7, which need a Linux toolchain and get one here natively. A single host is what makes the results in this paper comparable with each other rather than only within sections.

Two properties of it are measurement decisions rather than description, and a third is inherited from `pyperf`'s own tuned state. First, the frequency governor is fixed at `performance` and turbo is *disabled*: a turbo clock is by construction not constant — it depends on temperature, on how many cores are busy, and on how long the load has lasted — and a study whose conclusions include "these two configurations are indistinguishable" cannot rest on a clock that drifts. The cost is absolute speed, and it is paid deliberately. Second, the CPU has cores of two performance classes, so every single-threaded benchmark is pinned to the performance class; without that, the same benchmark can be timed on one class in one worker process and the other class in the next, and the difference lands in the dispersion. The thread-scaling study of §8 is the one exception, since it sweeps more threads than that class has cores. Third, address-space randomisation is left *on*, as `pyperf`'s own tuning advice requires: the argument for measuring across many processes is that layout, allocator state and hash seed differ between them, and disabling randomisation would remove exactly the variance that argument depends on. Table 1 reports all three as `pyperf` observed them at run time, not as we intended them.

Timing protocol. Every timed number in this paper is produced by `pyperf` [1], whose protocol we use as it comes: for each benchmark it calibrates an inner loop count until one value takes at least 100 ms, then runs **twenty independent worker processes**, each of which discards a warm-up value and keeps three, giving sixty values per benchmark. Not every suite can afford twenty in one go: where one value is a kernel of milliseconds to seconds the count is ten. The compute, flag and branchy suites are then run twice with the benchmark order reversed and both passes appended into one file — thirty values per pass, sixty in the file,

from twenty processes, and a direct check that the two passes agree; the CinderX, Static Python, collector-scale, pipeline and break-even suites are a single pass of ten processes: thirty values. Thread scaling, where one value is a full multi-second run, uses five processes in a single pass: fifteen values. On an interpreter that reports a JIT — PyPy and the `jit` build — `pyperf` switches itself to six processes of ten values, and we let it. The count is recorded with every result, and what matters is that it is always several separate processes, never one. Process-level repetition is the part that matters and the part a hand-written in-process timer cannot provide: address-space layout, allocator state, hash seed and code layout differ per process, and a dispersion figure computed inside one process does not include any of them. We report the median and the relative median absolute deviation.

`pyperf` also applies its own stability criterion — 95% confidence of under 1% variation — and warns when a benchmark misses it. That criterion is written for a tuned Linux machine and this host cannot hold a clock that steady. Across the 69 benchmarks of the compute suite the relative MAD is 0.2% at the median and 4.8% at worst, and about a fifth carry the warning. Rather than claim a criterion we did not meet, we state the resolution: differences above roughly 10% are resolved by these measurements, differences of a few per cent are not resolved by a pair of medians, and where the paper needs one it uses a significance test instead.

Two kinds of measurement need more than a callable. Workloads that consume or mutate their input, such as a garbage-collection pause or a pipeline stage, are registered through `bench_time_func`, where the rebuild happens inside the loop but outside the timed region, so the setup is never counted. Whole-process costs such as interpreter start-up and import latency go through `bench_command`, which times the process itself: a timer inside the process cannot observe the part of start-up that precedes it.

Where the paper says that two configurations do not differ, that is `pyperf`'s significance test (Student's t at 95%) over the two sets of values, not an inspection of two rounded numbers. The specialisation study of §6 runs at a *fixed* loop count of 200 rather than a calibrated one: it times a single function call of a few microseconds, and reaching `pyperf`'s 100 ms target would mean tens of thousands of fresh compilations inside one value. One value there is roughly 75 ms on 3.10 rising to roughly 103 ms on 3.14, of which roughly 1 % is the timed calls, so those numbers carry more relative noise than the rest of the paper and are read as a curve rather than as individual points.

Correctness gate. Kernels are defined once in Python (`kernels_py.py`, `branchy_py.py`); every other implementation must reproduce the reference value before it is timed. A mismatch is recorded as a result (`wrong_result`) instead of a time. The single exception is `fastmath`-style floating-point reassociation, which is allowed a 10^{-3} relative tolerance with the observed deviation stored alongside the timing; §4.1 reports it explicitly.

What is held constant. Within a research question exactly one factor varies: the implementation technology (§4, §5), the compiler flag set of otherwise identical C sources (§4.1), the interpreter version (§6), the runtime and its features (§7), the GIL configuration (§8), or the interpreter's own build flags (§9). Data layout is treated as part of an implementation: pure Python uses lists, the compiled and native implementations use contiguous `float64` buffers, exactly as an engineer adopting them would. Where the conversion cost between the two matters, it is measured and reported separately (§5, §10).

Machine drift. A machine that moves while it is measuring turns elapsed time into an apparent effect, which is why the protocol above is not negotiable: a measurement that takes all of its samples consecutively cannot tell the two apart, and an early pass of ours did exactly that, as the list below of what the protocol caught records. `pyperf`'s design defends against per-process variation and not against drift, and the two are easy to conflate. The worker processes remove everything that varies *per process* — the four properties listed in the timing protocol above — and that is a confound a single-process timer cannot even see. They do *not* spread a benchmark over the run: `pyperf` finishes all workers of one benchmark before starting the next, so two benchmarks compared across a long run are still separated in time, and a machine that is slowing down will show that separation as an effect.

Two things address the part that remains. Every suite except the CinderX, Static Python and Codon ahead-of-time runs registers a *machine probe* — a fixed native compute loop called through `ctypes`, whose cost does not depend on the interpreter — timed by `pyperf` like any other benchmark, so comparing it between two runs is a direct test of whether the machine changed, with the same significance test as everything else. And where a comparison turns on a few per cent, the two configurations are measured *alternately* rather than one after the other: short invocations, order swapped each round, appended into one file each (`bench/b1_compute/ab_flags.sh`), so both sample the same stretch of time. That protocol exists but no number in this paper is derived from it: §4.1's drift control is the two-pass reversed plan above.

Statistics and noise. Medians with relative MAD, rather than means with standard deviations, because the distributions are right-skewed by scheduler noise. Every run was made on an idle, mains-powered machine with no other workload — which this study learned to check rather than assume, at the cost recorded in §13. Absolute numbers are specific to this machine and to its fixed, turbo-disabled clock; what this paper reports are ratios.

How to read the multipliers. "$N\times$" means a *speed-up* throughout: how many times faster the first thing named runs, so $N > 1$ reads as a gain and $N < 1$ as a loss. The one exception is flagged where it occurs, by the phrase "at $N\times$ the time of" such and such a configuration: there the number is a fraction of a time, and smaller means faster. What a technique costs is given as a slowdown ("1.14× slower") rather than as a fraction of throughput, so that the direction reads off the notation itself.

What the protocol caught. Three concrete errors were found by the protocol rather than by inspection, which is the argument for having one. (i) An early pass took its samples consecutively per configuration and produced a clean, plausible and *false* result: pure-Python kernels appeared to slow down monotonically from 3.11 to 3.14, purely as a function of position in the run. (ii) The parallel variants of the pipeline in §10 produced a checksum that differed from the reference and differed between runs — a data race on a `ctypes` output cell shared by four threads. A timing-only benchmark would have reported the racing variant as a speedup, because losing updates makes the work smaller; the correctness gate rejected it before it was ever timed. (iii) The first specialisation-warm-up measurement produced a flat curve on every version, because one code object was compiled and then reused across repetitions, so only the first repetition was ever cold. Compiling per repetition, and replacing the loop body with straight-line code so that each instruction executes once per call, made the effect visible (§6).

4 RQ1: how much do JIT and native code buy on compute-bound kernels?

Question. How much does leaving the interpreter buy on a compute-bound kernel, and how much of that depends on which technology you leave it with? Six technologies are available to an engineer for the same loop — ahead-of-time compilation, two JITs, two foreign function interfaces and a vectorised library — and the decision between them is usually made without a like-for-like number for any of them.

Setup. Three kernels: a sequential sum of 10^6 `float64` values (the canonical example), a 200×150 mandelbrot escape-time grid with 30 iterations (nested loops, floating point, a data-dependent inner `while`), and a naive 96×96 `float64` matrix product (three nested loops, strided access). The implementations: a pure-Python loop, the builtin `sum()` where applicable, NumPy, Cython [7], Numba [6] (with and without `fastmath`), Codon [8] along both of the routes it offers — an extension module built ahead of time by `codon build -pyext` and the same kernels under `@codon.jit` — the same C kernels through both `ctypes` and a pybind11 extension module, and Rust through `ctypes`. All are timed on CPython 3.14.6; identical results are required before timing. The C, C++ and Cython extensions are compiled by the same `clang` 18.1.3 at `-O3 -march=native` and the Rust cdylib with `target-cpu=native` — which is the default recipe, and which §4.1 shows is not a neutral choice on one of these kernels.

Result: the compiled routes agree, and the library and the flag do not. Table 2 and Figure 1 give the measurement. On the array-sum kernel — summing 10^6 `float64` values — the pure-Python loop takes 32.23 ms and the builtin `sum()` 6.25 ms — 5.2× for the one change that needs no build step at all — while *every* statically compiled implementation lands in a band 0.31 % wide: Cython 31.9×, Numba 31.9×, C through ctypes 31.8×, C through pybind11 31.9×, Rust 31.8×, Codon ahead of time 31.9× and the same with pointer arguments 31.9× (1.009 to 1.012 ms). Inside a band this narrow the significance test resolves some pairs and declines others — Numba against C at $t = -3.79$ and the two C bindings against each other at $t = 3.30$, but Cython against C at $t = -1.47$ and against Rust at $t = -0.25$ — and a tenth of a per cent is below the resolution stated in §3. This kernel does not order the compiled routes. What it shows is the negative result, and the reason for it: every one of these implementations compiles to the same serial chain of additions, in which each add waits for the one before, so the language decides nothing. What does decide is permission to break that chain — the vectorised and fast-math rows below reach 2.5× this band on the same kernel. The grouping is looser on the mandelbrot grid (52.8–62.5×) and breaks apart on the matrix product (49.0–128.4×), for a reason that belongs to the compiler and is taken up below. The test separates the two C boundaries on the array sum as well, by a tenth of a per cent; the one place they separate by a

Table 2: RQ1: compute-bound kernels on CPython 3.14.6, on the single host of Table 1. Median per-call time and speedup over the pure-Python loop. NumPy's `matmul` row is a vendor-BLAS data point, not a compiler one.

	array sum (10^6 f64)		mandelbrot 200×150		matmul 96×96	
implementation	ms	×	ms	×	ms	×
CPython loop	32.23	1.0	46.92	1.0	73.27	1.0
CPython sum()	6.25	5.2	–	–	–	–
Cython	1.01	31.9	0.751	62.5	1.01	72.3
Codon AOT	1.01	31.9	0.865	54.3	1.50	49.0
Codon AOT (ptr)	1.01	31.9	–	–	0.587	124.7
Numba	1.01	31.9	0.864	54.3	0.571	128.4
Numba fastmath	0.419	76.9	0.683	68.7	0.535	137.0
Codon JIT	1.05	30.6	0.910	51.5	1.59	46.0
C (ctypes)	1.01	31.8	0.776	60.5	1.01	72.2
C (pybind11)	1.01	31.9	0.752	62.4	1.01	72.2
Rust (ctypes)	1.01	31.8	0.888	52.8	0.830	88.3
NumPy	0.411	78.4	9.98	4.7	0.049	1495.9

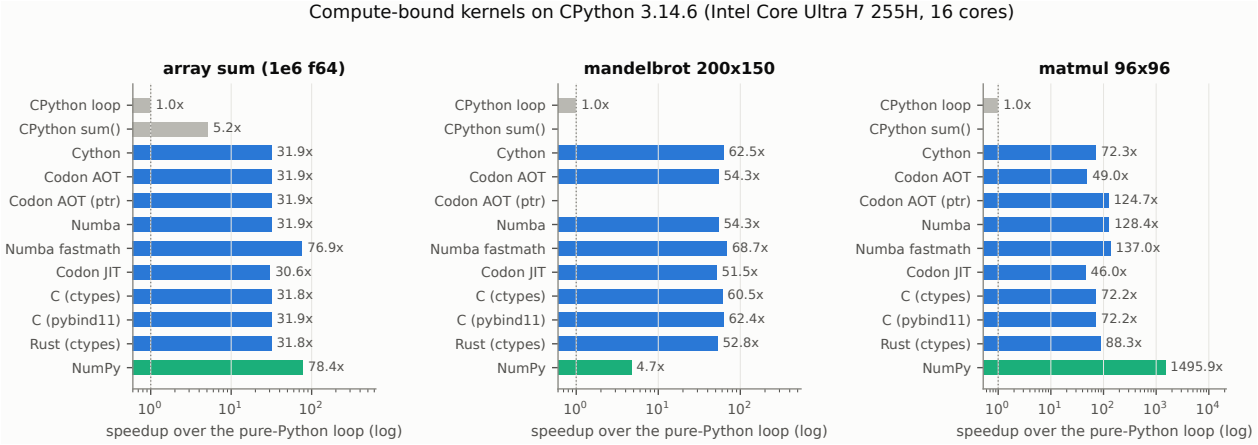

Figure 1: RQ1: measured speedups over the pure-Python loop (log scale). Every bar is a median over sixty `pyperf` values from twenty worker processes.

margin worth seeing is mandelbrot, by 3.3% ($t = 48.3$) — same source, same flags, two link paths, and a useful reminder of how large "the same program, twice" can measure.

Four observations are more interesting than the headline number.

First, **the plateau is not set by the language**. Numba with `fastmath` reaches 76.9× on the array sum — 2.41× beyond the compiled plateau — and the same C source compiled with `-ffast-math` reaches 0.418 ms, i.e. 77.1× (§4.1). Nothing about the language changed; what changed is the permission to reassociate floating-point additions, which breaks the serial dependency chain of the accumulator. §4.1 shows the factor belongs to that chain rather than to any compiler, by reproducing it at unchanged flags.

Second, **a library beats a compiler where a library exists**. NumPy is 78.4× on the array sum (a pairwise-summation kernel that is itself vectorised) and 1495.9× on the matrix product, where it dispatches to a vendor BLAS — three orders of magnitude beyond the naive triple loop that all other implementations faithfully reproduce. Conversely NumPy is the *slowest* accelerated option on mandelbrot (4.7×), because an escape-time loop with per-pixel early exit has to be expressed as full-grid masked passes. The same technology is thus 1495.9× on one kernel and 4.7× on another; a single "NumPy is fast" statement carries no information.

NumPy is also the only implementation in this comparison that is not single-threaded: its matrix product dispatches to a BLAS that takes as many threads as the affinity mask allows, while Cython, Numba, C and Rust get one core. Leaving that unstated would make the row a comparison of several cores of vectorised BLAS against one core of everything else, so the suite measures a second NumPy row, `numpy_1t`, which is the same code with `threadpoolctl` holding the BLAS to one thread outside the timed region. The two rows are statistically indistinguishable on two of the three kernels: array sum 0.4111 against 0.4111 ms (1.0001×, $t = 0.49$, not significant) and matrix product 0.0490 against 0.0488 ms (0.9966×, $t = 1.66$, not significant). Mandelbrot is the one exception, 9.98 against 9.96 ms (0.9982×, $t = 2.07$), a difference the test resolves and one that is two tenths of a per cent. At these problem sizes threading contributes nothing measurable, so the

plain `numpy` row is a like-for-like comparison.

Third, Numba is 128.4× on the matrix product against 72.2–72.3× for Cython/C/pybind11 and 88.3× for Rust. This is the one place in RQ1 where the compiled routes genuinely diverge, and the reason belongs not to the technology but to what the three trailing rows are built with: `-O3 -march=native`. On this kernel that flag is a regression, which §4.1 measures and which accounts for the whole gap. The apparent JIT advantage here is a property of the build recipe, not of just-in-time compilation.

Fourth, Codon is the one technology here whose result depends more on how the kernel is written than on which technology it is. Its two rows on the matrix product are 49.0× and 124.7× — a factor of 2.55 between them — and the source differs by three lines: whether the loop indexes the array or a pointer taken from it once. Codon has no way to state in a signature that an array is contiguous, the way Cython's `double[::1]` does, so indexing an `ndarray` goes through strides that are values at run time, and every element access carries that multiply-and-offset chain; the pointer form removes it. The idiomatic row therefore loses to Cython (72.3×) while the pointer row beats it and every native row in the table. That spread is larger than the gap between any two *languages* here: on this kernel the choice that mattered was one line inside one function. Both rows compute the reference result exactly, so nothing in the correctness gate distinguishes them.

The two Codon routes also price the JIT itself. Ahead-of-time and in-process compilation of identical source differ by 4–6 % (1.010 against 1.054 ms, 0.865 against 0.910 ms, 1.497 against 1.591 ms), which is the cost of the bridge rather than of the code it generates. What separates them is latency: getting one kernel running, from an imported accelerator to a compiled function, costs 1.33 s under Codon's JIT against 215 ms under Numba. Codon's share splits into 0.55 s to decorate the function and 0.79 s for the first call on a signature, none of it cached on disk, and the decoration is charged per function rather than once. §4.2 turns that into break-even call counts and reports separately what loading each accelerator costs, where the two differ by considerably more. The ahead-of-time route pays none of it, which is what makes `-pyext` the mode to prefer and Cython the appropriate comparison.

Finding. Leaving the interpreter is worth one to two orders of magnitude on compute-bound kernels. Which route you leave it by matters much less than that, and on the scalar reduction it barely matters at all: eight compiled implementations inside 4.4 %, and seven of them inside 0.31 %, two of those provably the same program. Where the routes do separate they separate by 2.79× on the matrix product, and in neither case is the cause the technology named on the row: at one end a compiler flag that is a regression on this kernel (§4.1), at the other a single line deciding how much address arithmetic the inner loop carries. So these measurements do not decide among Cython, Codon, Numba, C and Rust; what would decide it — build complexity, debuggability, code ownership — is not something this study measures. What they do say about that choice is that Codon's 2.55× spread against itself is wider than the spread between any two of the others, so "which technology" is the wrong granularity for that one. The performance choice one row below is the floating-point contract, worth a further 2.4×; and the row above is "does a library already do this", worth up to 1495.9×.

4.1 RQ1b: is the win the language or the compiler?

Question. RQ1 finds the choice among four compiled technologies worth far less than the decision to compile, which raises the question of what the compiler contributes at all. If code generation for the target processor is what produces the win, then one C source compiled at different optimisation levels and for different architectures should separate in the same way. Does it, and if a set of flags produces no difference in time, is that a measurement artefact or a property of the compiler?

Setup. One C source (`kernels_c.c`) is compiled six ways — `-O0`, `-O2`, `-O3`, `-O3 -march=native`, the same with the vectoriser disabled (`-fno-vectorize -fno-slp-vectorize`), and the same with `-ffast-math` — and all six are loaded through `ctypes` in one process, with Numba measured on the same data with and without `fastmath`.

Before timing anything, we ask what the flags actually changed. `codegen_diff.sh` disassembles each kernel out of each of the six shared libraries, normalises addresses and branch targets, and hashes the instruction stream; Table 3 is the result. Two variants that share a hash are the same program, and any difference in their measured time is then a statement about the machine, not about the flags.

Result: two of these flags are not variables, and one is a large one. Figure 2 summarises all three panels of what follows.

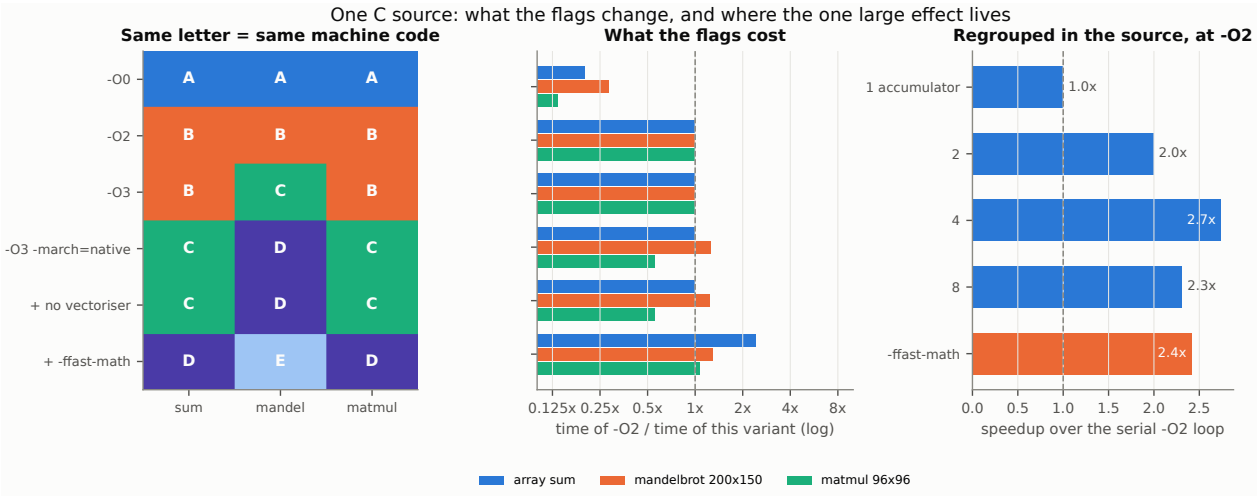

Figure 2: RQ1b: one C source, six flag sets, grouped by the machine code they actually produce, plus the accumulator ladder.

Table 3: Which flags change the emitted code. Variants joined by "=" are byte-identical machine code for that kernel, at the same address in the image (`results/codegen_identity.json`).

kernel	distinct programs among the six flag sets
`c_arraysum`	`-O0` | `-O2` = `-O3` | `native` = no-vectoriser | `-ffast-math`
`c_mandelbrot`	`-O0` | `-O2` | `-O3` | `native` = no-vectoriser | `-ffast-math`
`c_matmul`	`-O0` | `-O2` = `-O3` | `native` = no-vectoriser | `-ffast-math`

`-O3` is not a variable. On the array sum and the matrix product it emits the same instructions as `-O2`, in the same order, at the same address (38 and 92 instructions respectively), so equal run times are the only possible outcome; on mandelbrot it is a different program by one instruction (68 against 67) and still not a different time (0.9993×, $t = 0.81$, not significant). The reason is visible in the source: clang's `-O3` differs from `-O2` mainly in inlining and unrolling thresholds and a few loop transformations, none of which apply to three self-contained loops. What the identity map also buys is a resolution scale: on the matrix product the byte-identical `-O2` and `-O3` builds measure 0.5639 against 0.5633 ms — 0.11 % apart, and `pyperf` calls that difference significant ($t = 4.11$). One program, two labels, a significant difference: that is what 0.1 % is worth in this harness, and it bounds how any small ratio elsewhere in this paper should be read.

Disabling the vectoriser is not a variable either, and for a reason worth stating. For all three kernels the no-vectoriser build hashes identically to the `-march=native` build it is derived from, and the times agree (0.9999×, $t = 0.53$; 1.0009×, $t = -0.49$; 0.9998×, $t = 0.69$; none significant). This is not because vectorisation is unimportant — it is because clang emitted no vector instructions for any of these three kernels at `-O3 -march=native` in the first place, so the switch has nothing to remove. The one build in this study that does vectorise the array sum is the fast-math one (9 vector operations in 41 instructions), and it gets there by being allowed to reassociate, not by being asked to vectorise.

The architecture flag *is* a variable, on all three kernels, and its sign is not constant. `-march=native` changes the emitted code everywhere: the array sum from 38 to 37 instructions, mandelbrot from 67 to 121 (floating-point operations 17 to 32), the matrix product from 92 to 96. What that buys differs per kernel: nothing at all on the array sum (1.0104 against 1.0106 ms, $t = 0.19$, not significant), 1.25× on mandelbrot ($t = 410$, significant), and 0.557× on the matrix product ($t = -646$, significant): naming the exact CPU made that kernel take 1.80× longer than plain `-O2` — 1.0124 against 0.5639 ms. The regression is not hypothetical: `-O3 -march=native` is what the C, pybind11 and Cython rows of §4 are built with, and it is the whole of the gap that section measures between Numba and the ahead-of-time paths on the matrix product.

`-O0` is the one level that costs on every kernel here: against `-O2` it is 5.0× slower on the array sum, 3.5× on mandelbrot and 7.4× on the matrix product.

Result: the one flag that matters is not really a flag. `-ffast-math` is worth 2.42× on the array sum, 1.03× on mandelbrot and 1.92× on the matrix product, all relative to the `-O3 -march=native` build it is added to. The instruction counts say what it did on the array sum: the `-O2` build carries nine floating-point operations and no vector operations, and the fast-math build ten floating-point and nine vector operations — it is the only variant of this kernel that vectorises at all, and it can only do so once reassociating the sum

is permitted. The effect is therefore a broken dependency chain rather than instruction selection — so it should be reproducible without the flag, and it is. `accum_c.c` writes the same reduction with 1, 2, 4 and 8 independent accumulators and is compiled at plain `-O2` with no fast-math anywhere:

accumulators in the source	ms	vs 1 accumulator
1	1.0101	1.00×
2	0.5065	1.99×
4	0.3689	**2.74**×
8	0.4368	2.31×
`-O3 -march=native -ffast-math`	0.4182	2.42×

The one-accumulator source is the serial `-O2` loop by another name, and measures as one (1.0005×, $t = -0.64$, not significant), which is the control this ladder needs. Four accumulators written into the source, at unchanged flags, are 1.13× *faster* than granting the compiler blanket permission to reassociate ($t = -72.75$, significant); eight are 1.18× slower than four and 4.5 % slower than the fast-math build ($t = 50.60$, significant). So the ladder both explains the flag and overtakes it, and it has an optimum — four here — that a blanket flag cannot express. The price is identical in both cases and it is not the flag: it is a different summation order, and the correctness gate records it as such (the ladder steps deviate from the reference by $2\text{–}8 \cdot 10^{-15}$ relative, the fast-math build by $3.1 \cdot 10^{-15}$).

Why this suite is measured twice. The order in which `pyperf` works through a suite (§3) separates the first benchmark registered from the last in time, and any drift of the machine is charged entirely to the ratio between them — with the baseline of every ratio in this figure being one of the first benchmarks registered. The suite therefore runs twice, the second time with the plan reversed, both passes appended into one file, so each benchmark is measured at both ends of the run. In this run the dispersion that results is a relative MAD of 0.05–0.32 % across this suite, and the byte-identical variants land within 0.11 % of each other — which is what separates the 1.80× architecture-flag effect above from noise, and leaves nothing at all to report about `-O2` against `-O3`.

Finding. Three things, in order of how much they change a decision. First, the optimisation level that dominates build advice is not a variable for kernels of this shape: `-O2` and `-O3` are the same program on two of the three, and where they are not, they are not a different time either — so a study that reports a timing difference between them is reporting its own noise. Second, the architecture flag is a real variable on this target and it is not a free win: on one kernel it is worth 1.25×, on another nothing, and on the third it costs 1.80×. "Build with `-march=native`" is therefore not advice but an experiment, and one that has to be run per kernel; the disassembly is what tells you whether there is anything to run it on. Third, the one lever worth several times the run time is the floating-point contract, and it belongs to the program rather than to the compiler: a larger factor than the flag (2.74× against 2.42×) is available at `-O2` by regrouping the accumulation in the source, which grants the permission exactly where it is wanted instead of everywhere. Numba and clang are two front ends onto the same LLVM, and they measure accordingly: on the array sum and the matrix product they agree to within about 1 % with and without fast-math (array sum `-O2` vs. Numba 0.9981×, $t = 4.13$; `-ffast-math` vs. Numba fastmath 0.9978×, $t = 4.59$; matrix product `-O2` vs. Numba 1.0091×, $t = -12.59$ — all significant at this resolution). On mandelbrot Numba is ahead of `-O2` by 1.12× and of `-O3 -march=native -ffast-math` by 1.10× with fast-math; against the target-matched `-O3 -march=native` build it is 1.11× behind, Numba targets the host CPU by default, so the ordering against `-O2` is not a like-for-like one; what the residual against the target-matched build measures is code generation between the two front ends, not a property of just-in-time compilation. One limit belongs with all of this: `-ffast-math` changed the mandelbrot iteration count by $4.6 \cdot 10^{-5}$ relative, which is precisely the trade being made.

4.2 RQ1c: what does the JIT cost before it pays?

Setup. A break-even call count is meaningless without an input size, so we measure the same array-sum kernel at 10^3, 10^4, 10^5 and 10^6 elements in one process, measure each JIT's one-off compilation separately, and compute $k^* = t_{\text{compile}}/(t_{\text{Python}} - t_{\text{JIT}})$ per size. Two JITs are measured, Numba and Codon. CinderX's is deliberately not here: its baseline is a different interpreter build, so the numerator and the denominator would come from different runtimes, and its JIT is not something a program opts into one function at a time — it compiles at a call-count threshold whether or not anyone wanted it to, which makes "how many calls until it repays" a question no reader can act on.

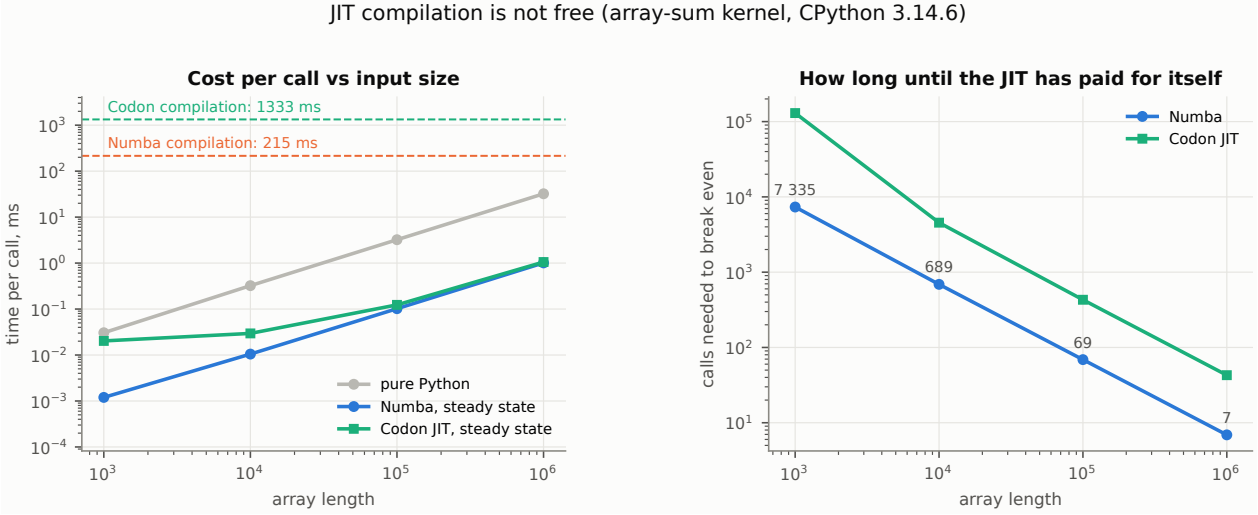

Figure 3: RQ1c: steady-state cost per call against input size for the pure-Python kernel and the two JITs, with each JIT's one-off compilation drawn as a horizontal line, and the resulting break-even call counts. Compilation here means decorating a function and compiling one signature; importing the accelerator is start-up and is reported separately.

Result: the break-even point moves by three orders of magnitude with input size. Figure 3. Compiling the signature under Numba costs 215 ms. Per-call times are 30.5 μs/1.2 μs at 10^3 elements, rising to 32.2 ms/1.01 ms at 10^6 — a steady 26–32×. The break-even point, however, moves by three orders of magnitude: **7335** calls at 10^3 elements, 689 at 10^4, 69 at 10^5 and **7** at 10^6. For a service that sums thousand-element arrays a few hundred times per request, Numba is a net loss unless the compilation is amortised across requests (or cached to disk); for the million-element case it repays itself almost immediately.

Codon's JIT reaches the same steady state and charges about six times as much to get there: 1.33 s against Numba's 215 ms, split into 0.55 s to decorate the function and 0.79 s for the first call on a signature. Both are measured from an imported accelerator to a compiled function; importing the accelerator is start-up, charged once per process rather than per kernel, and is reported below. Nothing Codon compiles is cached on disk, so a second process repeats all of it and there is no `cache=True` to reach for; the decoration is also charged per function rather than once. The break-even counts follow: $1.3 \cdot 10^5$ calls at 10^3 elements, 4545 at 10^4, 430 at 10^5 and **43** at 10^6 — six to eighteen times Numba's depending on size, and the reason is latency rather than generated code.

Its steady-state cost is not uniformly equal, either, and the gap runs the other way from the latency: at 10^6 elements Codon and Numba are within 5 % (1.054 against 1.010 ms), but at 10^3 Codon is 16.9× slower (20.3 against 1.2 μs). That is the JIT's call boundary, not its code — the same μs-scale crossing that the ahead-of-time extension does not pay, and it is invisible at the sizes where a JIT is usually argued about.

Compilation latency should not be read as a constant in either case. Numba's 215 ms is what the first `njit` compilation of this signature costs in a process that compiles nothing else, and it includes Numba's own lazy initialisation, which a second compilation does not pay. Codon's decomposes differently: the decoration is per function and the first call per signature, so "the compile latency" depends on which of the two a program is about to repeat.

Start-up, which is a separate cost. Importing the accelerator is not part of any break-even above, but it is not free and it is where the two differ most: `import numba` costs 190 ms and `import codon` costs 9.06 s. We attribute the bulk of the latter to Codon compiling its own native NumPy, which the bridge's initialisation imports unconditionally whether or not the program uses NumPy, and there is no supported switch to skip it. For a long-running service this is start-up latency paid once and then forgotten; for a command-line tool, a test runner or anything else that starts a fresh process per unit of work, it is nine seconds per unit of work, and it dominates everything else measured in this section. The ahead-of-time route pays none of it: the extension module built by `codon build -pyext` imports in 73 ms, like any other extension.

Finding. A steady-state ratio is only half of a decision. Whether a JIT is the right choice depends on the product of call count and per-call work, and on this host the threshold spans three orders of magnitude for one kernel and one accelerator — about 7 calls at 10^6 elements, about 7000 at 10^3 — and a further order between accelerators, from Numba's 7 to Codon's 43 on the same kernel at the same size. Any recommendation to use a JIT needs this second coordinate — and, before either accelerator repays anything, the cost of loading

Table 4: RQ2: branchy, allocating and irregular-access kernels on CPython 3.14.6.

	tokenize ($2{\cdot}10^6$ B)		binary trees (d=18)		BFS ($5{\cdot}10^5$ nodes)	
implementation	ms	×	ms	×	ms	×
CPython loop	300.44	1.0	126.81	1.0	203.95	1.0
Cython	24.25	12.4	25.86	4.9	20.54	9.9
Codon AOT	25.75	11.7	7.37	17.2	21.90	9.3
Numba	26.11	11.5	32.36	3.9	23.17	8.8
Codon JIT	26.17	11.5	7.32	17.3	23.03	8.9
C (ctypes)	22.29	13.5	7.05	18.0	20.62	9.9
Rust (ctypes)	22.42	13.4	6.09	20.8	22.38	9.1

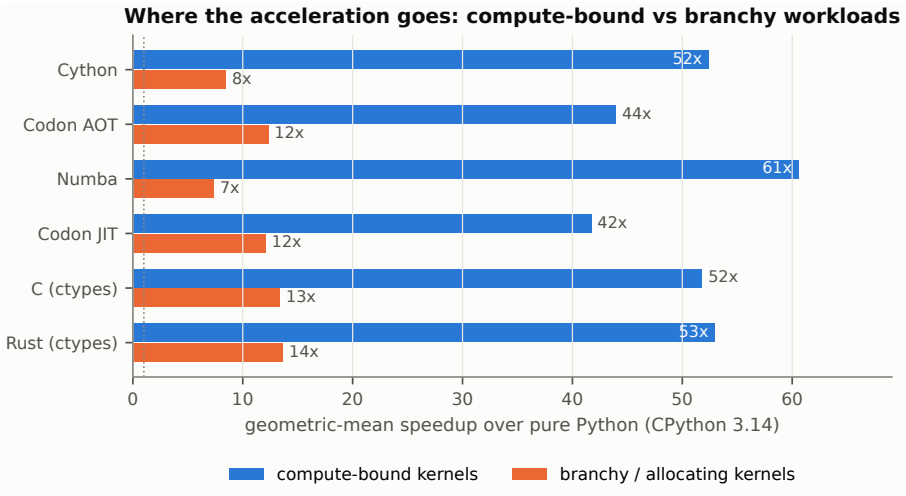

Figure 4: RQ2: geometric-mean speedup of each technology over pure Python, on the three compute-bound kernels versus the three branchy/allocating ones.

it at all, which is 190 ms for one of these two and nine seconds for the other.

5 RQ2: do branchy and allocating workloads really need C?

Question. Irregular code — many conditional branches, or structures reached by pointer — is where compilation returns least. Which property is responsible: the branches, the allocation, or the access pattern? And what does moving such code behind an FFI return, once the cost of the boundary itself is counted?

Setup. Three kernels isolate three different difficulties: *tokenize* is branch-heavy but allocation-free (a byte classifier with unpredictable branches over $2 \cdot 10^6$ bytes); *binary trees* is allocation-heavy (build and fold a depth-18 tree: $2.6 \cdot 10^5$ small objects, recursion, pointer chasing); *BFS* is irregular-access-heavy (breadth-first search over a $5 \cdot 10^5$-node graph with arithmetically generated edges). Each technology gets its best shot: Numba is given NumPy-typed rewrites and, for the tree, a `jitclass` node type; Codon is measured along both of its routes, and its tokeniser receives the same borrowed `uint8` view of the input that Cython and Numba get rather than the `bytes` object, which would arrive at the JIT as a generic Python object and be read through the CPython API.

Result: the three irregular kernels do not behave alike. Table 4 and Figure 4. Irregular workloads do resist compilation in aggregate: the geometric-mean speedup over pure Python drops from 42–61× on the compute-bound kernels to 7–14× here. The aggregate hides the interesting part, though, because the three kernels do not behave alike, and the property that separates them is not the one the phrase "branchy code" points at.

A compiler handles unpredictable branches over a typed array perfectly well. On *tokenize* — the most branch-heavy kernel, with a data-dependent state machine over 2 MB of pseudo-random bytes — all six compiled implementations land within 18 % of each other: C 13.5×, Rust 13.4×, Cython 12.4×, Codon 11.7× ahead of time and 11.5× under its JIT, Numba 11.5×. They are close rather than equal, and the pair

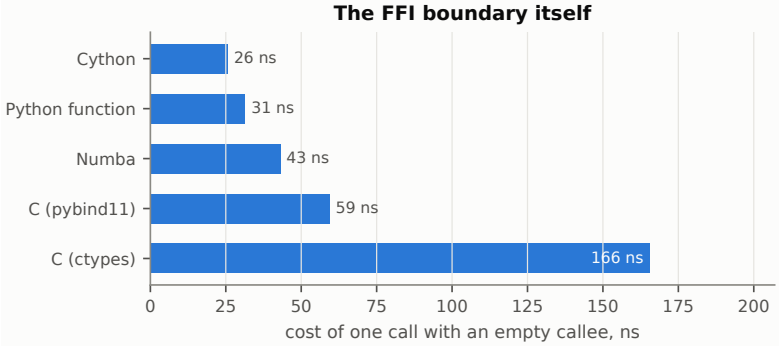

Figure 5: RQ2: the cost of crossing each boundary once, with an empty callee.

tests separate all fifteen pairs (Numba vs. C 0.8537×, $t = 123.9$; Cython vs. C 0.9192×, $t = 98.5$; Codon vs. C 0.8656×, $t = 164.9$; C vs. Rust 1.0060×, $t = -9.53$). The point is the size of the gap, not its existence — at most 17.4 % against the 13× that compiling the loop is worth.

Allocation is the obstacle, and the property that decides it is who owns the memory. On *binary trees*, which allocates $2.6 \cdot 10^5$ small objects per call, the six compiled implementations split cleanly in two: Cython 4.9× and Numba 3.9× on one side, Codon 17.2× ahead of time and 17.3× under its JIT, C 18.0× and Rust 20.8× on the other — a factor of 3.5–5.3 across the divide, and the pair tests separate it in every direction (3.51× Codon over Cython, 4.39× over Numba, both at $|t| > 240$; Codon 4.4 % behind C at 0.9563×, $t = 9.64$).

The line falls at memory ownership rather than at "native versus Python". Cython's node type is a `cdef class` and Numba's is a `jitclass`; both are typed, and both are still reference-counted objects that CPython's allocator hands out. Codon's is an ordinary class with two `Optional` fields — Python source, no `jitclass`, no flattening into arrays — and it lands with C and Rust because a Codon class is a native heap object with no reference count behind it. Typing the nodes is not what matters; replacing the allocator is, and that does not require leaving Python-shaped source.

Codon is also the control for the rival explanation. If the divide were *when* the code is compiled rather than what it allocates, its two rows would part company; they do not, at 7.37 against 7.32 ms, a difference the significance test does not resolve.

Irregular memory access is not an obstacle either: on *BFS* over a $5 \cdot 10^5$-node graph, Cython (9.93×) and C (9.89×) are within half a percent of each other — the difference is significant but tiny (1.0042×, $t = -4.02$) — with Codon (9.31× and 8.86×), Rust (9.11×) and Numba (8.80×) close behind.

The obvious explanation is that all six abandoned the deque for a preallocated array, so that what is being measured is the data layout rather than the language. We measured that explanation instead of assuming it, because every compiled implementation here changes both things at once. `bfs_py_flat` is the same traversal in pure Python with the queue as a preallocated list — same algorithm, same visiting order, same result — and it takes 228.5 ms against the deque version's 204.0 ms: the flat layout is 1.12× **slower** ($t = 60.74$, significant). Flattening the data structure, by itself and in pure Python, is a regression on this kernel; the entire 9.9× is the cost of interpreting the loop. The rule "flatten your data structures first" is not supported here; what is supported is that a flat layout is what *lets* the loop be compiled, and the compilation is where the factor comes from.

Three smaller differences are worth recording because they cut across RQ1b rather than across languages. C and Rust, compiled from the same algorithm at equivalent optimisation levels, differ by less than 1 % on tokenize (Rust 0.6 % slower, significant), by 1.16× in Rust's favour on binary trees, and by 0.92× against it on BFS. Differences of that size between two LLVM-based toolchains are code generation, not a property of either language, and they are smaller than the flag effects §4.1 measures inside one compiler.

The tokeniser is worth one more look, because it is the kernel where the two could plausibly diverge: it is a state machine over unpredictable input, so the interesting question is whether a compiler emits a chain of conditional branches, which mispredicts, or branchless conditional selects, which do not. Table 5 disassembles both. The branch shape is identical — twenty conditional branches and six conditional selects each — even though the instruction counts are 167 against 180. The 0.6 % they do differ by is therefore not branch prediction: the difference is real, but it is not where it would be most natural to look for it.

The boundary itself. Figure 5 measures one call with an empty callee: Cython 26 ns, a plain Python function 31 ns, a Numba dispatcher 43 ns, a pybind11 function 59 ns, and `ctypes` 166 ns. Two consequences. First, an FFI call is not free but it is cheap: at $\approx 10^2$ ns, a kernel doing $\geq 20\,\mu$s of work pays under 1% for the crossing even across `ctypes`, and $\geq 6\,\mu$s is enough for the cheaper bindings. Second, `ctypes` — the most

Table 5: RQ2: the tokeniser disassembled out of both shared libraries. The mnemonics counted are architecture-specific and are recorded, with the architecture, in `results/codegen/summary.json`.

tokenize kernel	instructions	cond. branches	cond. selects	ms
C, clang -O3 -march=native	167	20	6	22.29
Rust, rustc -C opt-level=3 -C target-cpu=native	180	20	6	22.42

Disassembled on x86_64; the branch and select mnemonics counted are recorded in `results/codegen/summary.json`.

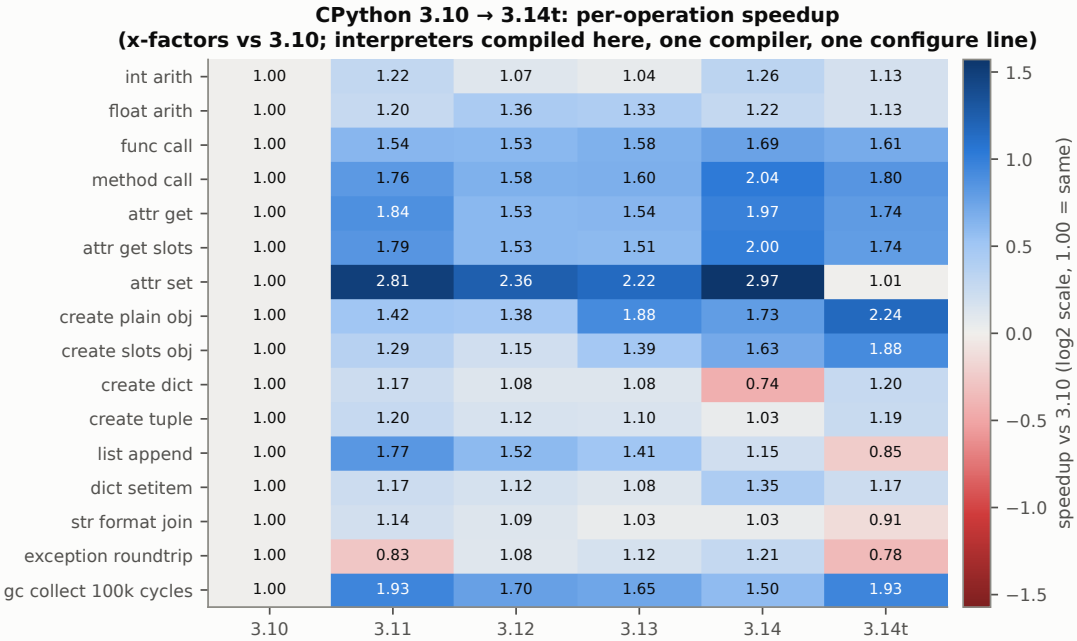

Figure 6: RQ3: per-operation speedup relative to CPython 3.10 (values are × factors; blue = faster, red = slower).

convenient way to call native code — is the most expensive boundary, 5.3× a Python-level call, so per-record FFI calls are a real cost: the 20 000 per-record `ctypes` calls in §10 spend ≈ 3.3 ms on boundary crossings alone.

Finding. The class needs splitting in two, because the discriminating property is *who owns the memory*, not how many branches the code contains. A branch-heavy scan over flat data does not need C: Cython, Codon and Numba all reach within 18 % of it. An allocation-heavy object graph is where C and Rust lead Cython and Numba by 3.7–5.3×, and what closes that gap is replacing the object model rather than typing it. Codon shows this can be done without leaving Python-shaped source: 17.2× against C's 18.0×, from a class with two `Optional` fields. The recommendation to push allocation-heavy code behind an FFI is therefore supported only in its weak form — such code needs an allocator that is not CPython's, which is not the same claim as needing C.

6 RQ3: what did CPython 3.10–3.14 do for object-heavy code?

Question. Specialising bytecode, call inlining, immortal objects and allocator work all arrived in CPython between 3.10 and 3.14. What did they deliver for object-heavy code, which release delivered it, and is the delivery uniform across operations or concentrated in a few?

Setup. A stdlib-only suite (so that the interpreter is the only variable) run on CPython 3.10.18, 3.11.6, 3.12.12, 3.13.5, 3.14.6 and free-threaded 3.14.6t: instance creation for dict-based, `__slots__`-based, plain-dict and tuple representations; attribute get/set; bound-method and plain-function calls; integer and float arithmetic loops; list/dict/string operations; exception round-trips; a garbage-collection pause over 10^5 objects in reference cycles; plus non-timing structural probes (reference counts of singletons, allocator blocks per instance, startup latency).

Table 6: RQ3: structural runtime properties measured by the same suite.

property	3.10	3.11	3.12	3.13	3.14	3.14t
`sys.getrefcount(None)`	7 780	7 887	4 294 967 295	4 294 967 295	3 221 225 472	3 221 225 472
None is immortal	no	no	yes	yes	yes	yes
alloc. blocks / instance (dict)	5.96	4.96	4.96	3.96	3.96	3.96
alloc. blocks / instance (slots)	3.96	3.96	3.96	3.96	3.96	3.96
max RSS after suite MB	27	28	30	28	30	36
bare startup ms	9.0	9.4	10.5	10.6	10.6	13.7

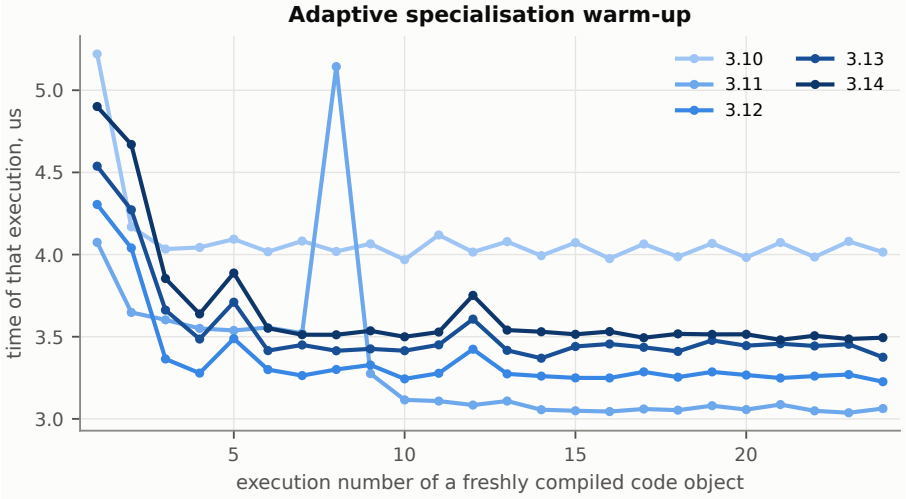

Figure 7: RQ3: per-execution time of a freshly compiled code object over its first executions, on each release. Each point is a separate `pyperf` benchmark whose timed region is that one execution.

Why the builds had to be made here. A version comparison is only a version comparison if nothing else changes with the version, and an interpreter's build options are worth as much as several releases of interpreter work (§9). Every version in this section is therefore compiled from its release tarball by `bench/build_uniform.sh` with one compiler and one configure line — `-enable-optimizations`, spelled identically in every release from 3.10 to 3.14 — so that the only difference between the six interpreters is the source they were built from. The free-threaded build differs in exactly one further token, `-disable-gil`, and is reported separately in §8.

Result: the gain is real and it did not arrive one release at a time. Figure 6, relative to 3.10. The geometric mean over the sixteen operations is 1.44× at 3.11, 1.35× at 3.12, 1.38× at 3.13 and 1.45× at 3.14:

- **3.11 is the step change**, as the specialising-interpreter work [10] predicts: attribute writes 2.81×, attribute reads 1.84×, `list.append` 1.77×, bound-method calls 1.76×, function calls 1.54×, cyclic garbage collection 1.93×. In aggregate it already reaches 1.44×, against the 1.45× that three further releases finally deliver.
- **3.12 gave part of it back**, and the giving-back is not marginal: integer arithmetic falls from 1.22× to 1.07×, attribute reads from 1.84× to 1.53×, attribute writes from 2.81× to 2.36×, `list.append` from 1.77× to 1.52×. Two operations improve at 3.12: float arithmetic, which peaks there at 1.36× and stays near it afterwards, and exception round-trips, which gain 1.30× over 3.11.
- **3.14 is the best release in the sweep for most object work.** It sets the peak on method calls (2.04×), attribute reads (1.97× dict-based, 2.00× slotted), attribute writes (2.97×), function calls (1.69×), integer arithmetic (1.26×), `dict` item assignment (1.35×) and exception round-trips (1.21×). Against 3.13 those are large single-release moves — attribute reads 1.29×, method calls 1.28×, attribute writes 1.34×, all significant — so the shape of the sweep is a step at 3.11, a dip through 3.12–3.13 and a second step at 3.14, not a plateau.
- **Not everything was recovered, and one operation ends below where it started.** Building a dict literal is 0.74× 3.10, i.e. 35 % *slower* (4.48 to 6.05 ms per 20 000 dicts), and the whole of that regression arrives at 3.14 (1.46× slower than 3.13, $t = -128$). `list.append` peaked at 3.11 (1.77×) and is 1.15× at 3.14; cyclic garbage collection peaked at 3.11 (1.93×) and is 1.50×.

- **The free-threaded build is a different profile, not a uniformly slower one.** Its geometric mean is 1.33×, but it is the best configuration measured for instance creation (plain objects 2.24×, slotted 1.88×) and matches 3.11 on garbage collection (1.93×), while attribute writes collapse to 1.01× — 3.10's number — and `list.append` (0.85×), exception round-trips (0.78×) and string formatting (0.91×) end below 3.10.

Which of the mechanisms are visible in the runtime itself. Table 6 contains the structural probes. Immortal objects (PEP 683 [12]) are clearly present from 3.12 — `sys.getrefcount(None)` jumps from $8 \cdot 10^3$ to $2^{32}-1$, and to 3 221 225 472 on 3.14 — yet 3.12 is *slower* than 3.11 on almost every timed operation. Immortality removes reference-count writes to shared singletons, which pays off under multi-threaded contention, not in single-threaded loops; our measurements simply do not exercise what it improves.

The allocator work, by contrast, is visible and explains a rule of thumb that has quietly expired. Allocator blocks per instance fall from 5.96 (3.10) to 4.96 (3.11–3.12) to 3.96 (3.13–3.14) for a dict-based instance, while a `__slots__` instance stays at 3.96 throughout: by 3.13 a normal object costs the allocator exactly what a slotted one does. In creation time the `__slots__` advantage shrinks from 25% (3.10) to 14% (3.11) to 4% (3.12) and actually *reverses* on 3.13, where slotted instances are 8% slower than dict-based ones (6.48 vs 5.98 ms per 20 000) — before returning to an 18% advantage on 3.14, where the structural probe says the two shapes still cost the allocator the same. The mechanism the rule is usually justified by — one allocation fewer per instance — therefore stopped applying at 3.13, while the difference in creation time did not: `__slots__` is still worth 18% on 3.14, for a reason these measurements do not identify. The memory row of Table 6 is the one that costs rather than pays: peak RSS after the same suite climbs from 27 MB on 3.10 to 30 MB on 3.14 and to 36 MB on the free-threaded build — a fifth more memory for the build measured in §8, charged whether or not a second thread ever starts.

Figure 7 was built to make the adaptive interpreter directly visible, and it does not. On a freshly compiled straight-line arithmetic function the first execution costs 1.30× the steady-state cost on 3.10, 1.33× on 3.11, 1.32× on 3.12 and 3.13 and 1.40× on 3.14 — and 3.10 has no specialisation to warm up. A first-execution penalty of that size is therefore present with and without the adaptive interpreter, so this probe cannot attribute it to specialisation; what it measures is the cost of running a code object for the first time, whatever the interpreter does with it. The same figure carries a cleaner result: the steady state itself improved sharply at 3.11, from 4.03 µs on 3.10 to 3.06 µs, and has since drifted back to 3.51 µs on 3.14, so straight-line integer code is 1.15× faster on 3.14 than on 3.10 once warm and 1.15× *slower* than on 3.11.

Start-up moved the other way across the sweep and belongs next to the throughput row: a bare interpreter start costs 9.0 ms on 3.10 and 10.6 ms on 3.14, importing the standard set of modules 47.6 ms against 66.3 ms, and the free-threaded build is slower again (13.7 and 75.8 ms). For a long-running service this is invisible; for a command-line tool invoked per file it is the only number in this section that matters.

Finding. Upgrading the interpreter is worth real money for object-heavy code: attribute and method operations are 1.97–2.97× faster on 3.14 than on 3.10, and the geometric mean over the whole suite is 1.45×. Three refinements come with that. The gain is not spread evenly across releases — 3.11 delivers most of it, 3.12 and 3.13 give part of it back, and 3.14 recovers it and adds more — so "upgrade" is good advice and "every release helps" is not. Of the mechanisms usually credited, our measurements attribute gains to the specialising interpreter and to the allocator; immortal objects are demonstrably present from 3.12 and demonstrably invisible to single-threaded code. Where they are supposed to pay — reference-count contention between threads — this study does not isolate them, so this row can say only that they are not what moved these numbers. And the row needs two warnings attached: one operation in our suite is 35 % slower on 3.14 than on 3.10 and got that way in the last release, and the stdlib-import start-up is 39 % slower across the same sweep — so an upgrade deserves its own measurement on the code that matters.

7 RQ4: Cinder and CinderX — which capabilities pay, and at what cost?

Question. Meta's Cinder fork and its CinderX extensions offer a JIT, a static typing mode with unboxed primitives, a parallel garbage collector, lightweight frames and lazy imports. Which of those pay off on ordinary code, which need the code rewritten, and what does the open-source state of the project cost an adopter?

Setup. Stock CPython 3.14.6 and Meta's Cinder fork (`meta/3.14`, 3.14.5+meta) are built from the vendored sources on this host with identical `configure` flags and one compiler (clang 18.1.3), so the delta is the Meta patch set alone; neither gets PGO. The CinderX extension package is then built against the fork — twice, because `ENABLE_ADAPTIVE_STATIC_PYTHON` gates the specialising path of CinderX's own interpreter and therefore

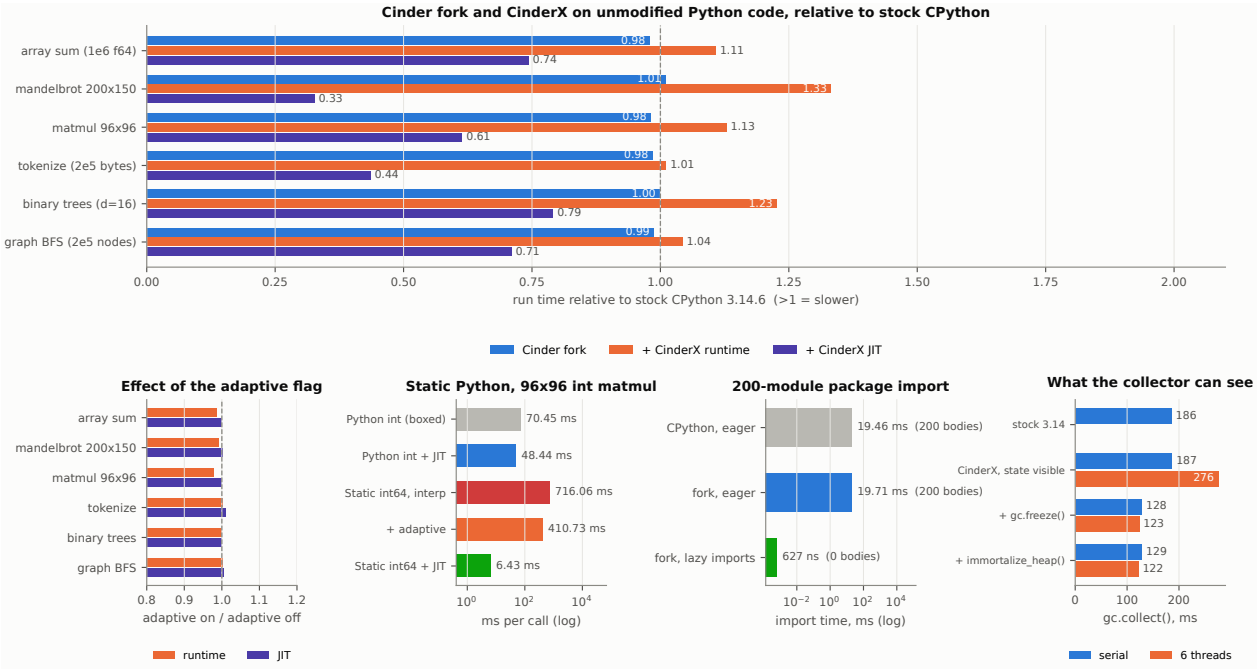

Figure 8: RQ4: (top) the fork, the CinderX runtime and the CinderX JIT on unmodified Python code, relative to stock CPython 3.14.6 built with the same flags; (bottom left) the effect of the adaptive Static Python build flag on those same ratios; (bottom centre) the Static Python `int64` matmul (plotted for the slow layout draw, 6.43 ms; the fast draw is 4.25 ms, §7), against the same kernel boxed and boxed under the JIT, and interpreted with and without adaptive specialisation; (bottom right of centre) import latency of a 200-module package with and without the fork's lazy imports; (bottom right) one collection of a worker's heap — long-lived state plus per-request garbage — with the state visible to the collector, after `gc.freeze()` and after `immortalize_heap()`, serial and on six threads, with stock CPython on the same heap for reference.

bears directly on performance. Every configuration below is measured with that flag off and with it on, each built into its own tree, both from one source in one pass. `ENABLE_EVAL_HOOK` is off in both, because it is the pre-3.13 mechanism that PEP 523 [13] replaced on 3.14 — off is the correct setting, not a limitation. Configurations are measured on the same six kernels as RQ1/RQ2, plus the fork's own features (lazy imports, parallel GC, frame handling) and Static Python.

How the JIT is asked to compile is part of the setup and not a detail, because it decides the answer. The JIT compiles the bytecode it is shown, and CPython 3.14 rewrites that bytecode as a function runs, so a function compiled before its first call is compiled from fully generic bytecode. Every JIT row below therefore uses the JIT's own threshold (`compile_after_n_calls`), and two things are then checked in the process that will do the timing: that the kernel is compiled, and that one further call does not increase `count_interpreted_calls`. The second check is the load-bearing one — see the end of this section.

Result: the fork itself is not the story. Built with identical flags, the Cinder fork lands within 2.1 % of stock CPython 3.14.6 on all six kernels (0.98–1.01× the stock time, Figure 8 top). Five of those six differences are resolved by the significance test and one — binary trees, 0.9996× at $t = 0.15$ — is not, so the honest statement is not "the same" but "measurably different and not usefully so": the largest of them is the array sum at 0.979× ($t = 5.2$), i.e. the fork is 2.1 % faster there, with the matrix product next at 0.982×, and nothing moving by more than 2.1 % either way. Its deep runtime features are opt-in, and the one that pays here is lazy imports: with them enabled, importing a 200-module package executes 0 module bodies instead of 200 and drops from 19.7 ms to 0.63 µs, with the first use of a deferred module costing 0.35 ms. That is a startup-latency feature, and stock CPython has no equivalent — our probe on stock is recorded as `unavailable` (`importlib.set_lazy_imports()` does not exist there) rather than as a slower time. The fork's lightweight-frame machinery is the one that does nothing useful here: a 200-deep call chain is 1.01× stock, a 50-frame traceback 0.97× and a 100-frame walk 1.01×; the test resolves all three, and not one of them is a difference worth having. The parallel collector gets its own paragraph below, because one number turned out to be the wrong shape of answer for it.

Result: the runtime costs, the JIT more than repays it, and the adaptive flag changes neither. Initialising the CinderX runtime — which installs its own frame evaluator in place of the one CPython

Table 7: RQ4: what the open CinderX actually delivers, built and run natively on the host of Table 1.

capability	verdict	measurement
builds on Linux x86_64	yes	0 errors in both configurations, with the patches carried in the vendored tree
JIT compiles & is correct	yes	all 8 kernel functions, compiled at the JIT's own threshold and verified to be what runs
CinderX runtime, no JIT	cost	matmul 1.13× the stock time, 1.01–1.33× over the six
CinderX JIT on untyped code	win	matmul 0.61× the stock time, 0.33–0.79× over the six
same, adaptive configuration	win	matmul 1.11× / 0.61× the stock time
Static Python + JIT	win	11.4× faster than the same kernel boxed under the same JIT (7.5× on the slow layout mode)
Static Python, interpreter	cost	10.2× slower than boxed Python
adaptive specialisation, once completed	win	static interpreter 716 → 411 ms; no effect under the JIT
lazy imports	win	19.7 ms → 0.63 μs; 0/200 module bodies run; first use 0.35 ms
parallel GC	composition-dependent	0.68× serial with the state visible, 1.06× after immortalising it (187 → 129 ms serial)
lightweight frames	no effect here	200-deep call chain 1.01× stock

specialises — makes the same kernels 1.01–1.33× *slower* than stock, worst on the mandelbrot grid. Under the runtime the frame probes cost more than the kernels do: a 200-deep call chain 1.37×, a 50-frame traceback 1.15×, a 100-frame walk 1.13×. That is the same machinery that was neutral on the bare fork, so the cost arrives with the extension rather than with the patch set.

The JIT then repays it with room to spare, on every kernel: 0.33–0.79× stock (Figure 8 top), i.e. 1.26–3.06× faster. The order is informative but not clean. The mandelbrot grid gains most (0.33×) and the binary trees, which allocate $6.6 \cdot 10^4$ small objects per call, gain least (0.79×) — what a method JIT removes is interpretive overhead, and allocation is not interpretive overhead. But allocation is not the only thing bounding it: the array sum gains nearly as little (0.74×) while allocating nothing, and graph BFS rather more (0.71×) while allocating a great deal. Where allocation does bind, §5 measures what moving the bound is worth: Codon, compiling comparable source but allocating its nodes natively, runs the binary trees 17.2× faster than pure Python where the CinderX JIT manages 1.26× — on a tree of depth 18 against RQ4's depth 16, so the two factors are indicative rather than commensurable. A JIT that keeps CPython's objects is working on the smaller half of that problem, and no amount of compilation reaches the other half. The frame probes are the one place it does not help uniformly: a 200-deep call chain 0.85×, a 50-frame traceback 0.98×, a 100-frame walk 1.33×.

The adaptive flag is not involved either way. Adaptive-on to adaptive-off is 0.98–1.00 under the runtime and 1.00–1.01 under the JIT (Figure 8, bottom left), and the stock-relative ranges are 1.01–1.32× and 0.33–0.79× against 1.01–1.33 and 0.33–0.79 without it. On untyped code the two configurations are the same interpreter.

Result: the defaults are the configuration. The JIT takes 37 documented options, through `-X cinderx-jit-...` or the environment, each with a one-line help string in the source and no other documentation. We measured all 37 one at a time, and eleven compositions chosen by how the options could unblock one another rather than by enumeration — a larger inlining budget together with the preload and cold-call limits that bound it, annotation guards together with that budget, the hot/cold code split together with the huge-page mapping it exists for. *Nothing is faster than the defaults.* The nine configurations that matter were then re-measured under this paper's own protocol. Figure 9 shows the ones a reader might plausibly change; a configuration that switches an optimisation off is slower, which is not a finding, so those are measured but not plotted.

The largest effect is the one nobody would guess: asking the JIT to compile *sooner* makes it slower. Compiling every function at its first call — the option is named `jit-all` — runs the six kernels at 0.49× the default configuration's speed, and 0.23× on the mandelbrot grid. What that setting loses is identifiable: forbidding the JIT to compile specialised opcodes gives 0.49×, and the two agree on every kernel to within 0.02. The penalty we first reported as a property of the JIT is therefore exactly one lost optimisation, and it is one `-X` flag away from any user. Nothing else a reader can turn on helps either: splitting hot from cold code gives 0.96×, the huge-page mapping it is designed to work with does not change that, and switching off inline caches for attribute access gives 0.97× overall but 0.87× on binary trees and 0.92× on graph traversal.

One group of options is missing from all of this and the reason is ours, not the JIT's. Four of the 37 tune

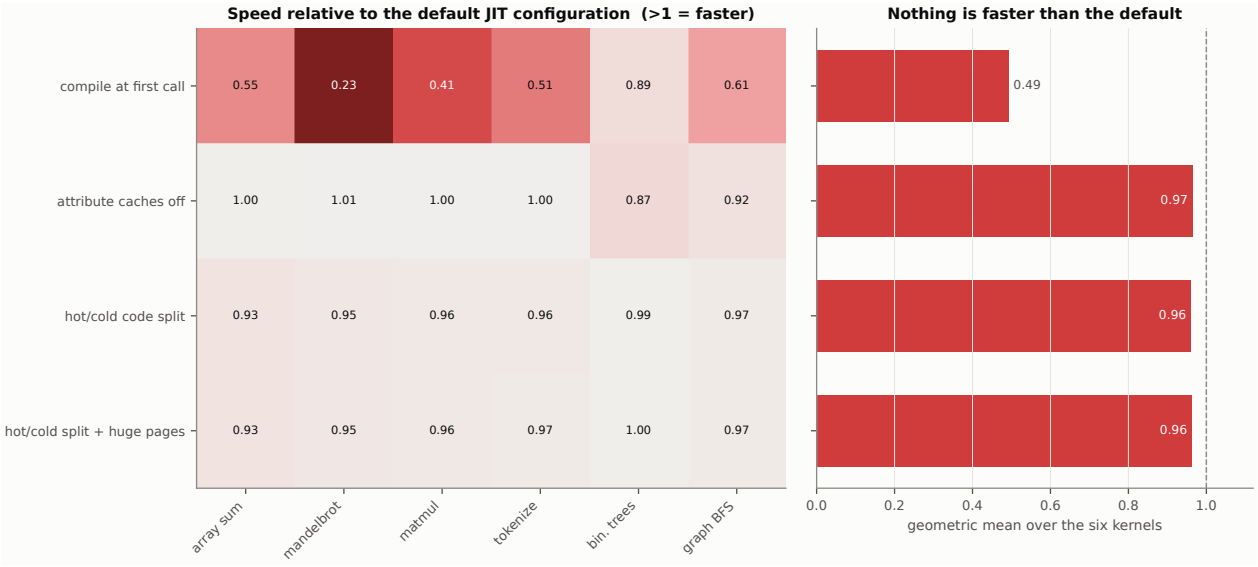

Figure 9: RQ4: the JIT settings a reader might actually change, against the configuration the rest of this section was measured in. Left, per kernel; right, the geometric mean over the six. The five configurations that exist only to identify which optimisation another setting loses — `no-jit`, `no-inliner`, `no-specialized-opcodes`, `no-simplifier` and `no-stable-frame` — are measured but not plotted here; the two of them that identify a loss are quoted in the text.

the inliner — its cost budget, how many dependent functions are preloaded with a compilation, when a call site is pruned as cold, how far the simplifier may run. On our six kernels the inliner has almost nothing to do: asked afterwards, it reports two inlined functions across the whole set, both in binary trees, and none at all in the other five, because their hot loops do not call Python functions. Raising the budget tenfold with the pruning gate open and the preload limit raised leaves those counts and the compiled sizes byte-identical. So the four options are inert here by construction, and that is a statement about the workload rather than about them: on the one kernel that does exercise the inliner, switching it off gives 0.91×, and raising its budget from there gives nothing, which is what a saturated optimisation looks like.

Result: the adaptive flag shipped with half of its mechanism, and we supplied the other half. On untyped code the flag changes nothing, as above. On the code it exists for it did worse than nothing: at the commit we started from, `ENABLE_ADAPTIVE_STATIC_PYTHON=1` meant Static Python did not run at all. The typed module imported and compiled, and the first execution of the primitive accumulation `s = s + a[i*n+k] * b[k*n+j]` raised `TypeError: 'int' object is not iterable`. Ordinary boxed Python on that same interpreter ran normally, so the failure was specific to the path the flag is supposed to accelerate.

Reporting that as the project's verdict would have been the cheap option and the wrong one, because "the feature is broken" and "the feature is half-ported" are different findings for someone deciding whether to adopt it. It is the second. The 3.14 interpreter carried twelve sites that specialise a static opcode — seven `_Ci_specialize`, five `specialize_with_value` — and no handler for any of the opcodes they specialise into; the 3.12 interpreter in the same tree has eight such handlers, so the mechanism existed and was carried across only in one direction. Three consequences followed, each independently fatal. `_Ci_specialize` wrote the cached opcode over the `EXTENDED_OPCODE` prefix rather than over the static opcode itself, and the low byte of a static opcode is an ordinary CPython opcode — `STORE_LOCAL_CACHED` is 16 and stock opcode 16 is `GET_ITER`, so the next execution of that instruction ran `GET_ITER` on the integer being stored, which raises exactly the `TypeError` observed. The static compiler reserved no inline-cache units for the opcodes whose specialisations write one, so `CAST` wrote its 32-bit cache over the two instructions that followed it. And the dispatcher stepped over exactly one unit regardless.

Completing it took handlers for the eleven reachable specialised opcodes, one shared table of how long a static instruction is — used by the interpreter, by the JIT's bytecode reader and by the disassembler, so that the three cannot disagree — cache reservations in the compiler's opcode metadata, and cache payloads written one byte per unit with the opcode byte left at `CACHE`. That last point is the subtle one: CPython walks a code object one unit at a time and cannot know that a static opcode reserves units behind it, so a raw 32-bit payload whose first byte happens to name a cache-carrying opcode sends `deopt_code` past the end of the code object. `INVOKE_INDIRECT_CACHED` is deliberately left unspecialised: caching a `PyObject**` would cost eight units at every `INVOKE_FUNCTION` site to serve the rare container that is immortal but not immutable.

The patches are in the vendored tree.

The verification matters more here than the patch, because an interpreter that quietly does not specialise would read as a null result rather than as a bug. CinderX's own `test_compiler/test_static`, 3910 tests, passes under `PYTHONMALLOC=debug` in both configurations; before the change the non-adaptive build *aborted* that suite and the adaptive one could not run Static Python at all. The suite's own discovery pass now reports three rows and no gate failure in both configurations, against the same reference checksum, and a probe reads `_co_code_adaptive` after execution and requires the cached opcode to be present — so the specialisations are shown to fire, not merely to be harmless.

Result: Static Python is where the design pays off — and only with the JIT. On a 96×96 integer matrix product, the same source compiled as Static Python with unboxed `Array[int64]` takes 716.1 ms under the CinderX interpreter against 70.4 ms for ordinary boxed Python on the same interpreter — a 10.2× *slowdown*. Bytecode inspection shows the lowering: 61 of 158 instructions are primitive or extended-opcode forms. Compiled, the same function takes 4.25 ms in most worker processes — and 6.4 ms in the rest, which needs stating before any ratio is drawn from it.

That row is the one place in this study where address-space layout reaches a headline number. Its samples are bimodal, 4.25 ms or 6.4 ms with nothing in between, and the mode is a property of the process rather than of the sample: across the sixty samples of the two build configurations, 33 landed in the fast mode and 27 in the slow one, every sample of a process agreeing with the others. The median is therefore not a statistic here — it reports whichever mode won the draw, and it came out 4.25 ms in one configuration and 6.43 ms in the other from identical binaries. Disabling randomisation collapses the effect: twelve runs under `setarch -R` all land at 4.25–4.26 ms. Nothing else in the suite behaves this way, including the same kernel boxed under the same JIT, whose thirty samples span 1.02×; it is the tight compiled loop that is placement-sensitive. This is what leaving ASLR on buys (§3): a fixed-layout measurement would have reported one of the two modes as the answer and nobody would have known there was another.

Against what, then, should 4.25 ms be read? "16.6× faster than boxed Python" is the comparison usually offered, and it credits the types with what the compiler did, because the boxed side of it is interpreted. The same boxed kernel under the same JIT takes 48.4 ms, so the JIT alone is worth 1.5× here and *the types are worth* 11.4× *on top of it* — or 7.5× for a process that drew the slow layout. That is the number an adopter needs, and it is a range rather than a point.

The adaptive path, now that it runs, is worth a large fraction of the interpreted cost and none of the compiled one: static interpretation drops from 716.1 ms to 410.7 ms, a 43 % saving, while the compiled row is unmoved. That is the expected shape — the flag specialises interpreter dispatch, and the JIT does not dispatch — and it settles what the primitives are for. Even fully specialised, interpreting them is 5.8× slower than not using them — and 8.5× slower than handing the same untyped source to the JIT — and the reason is visible in the encoding: every primitive operation is an `EXTENDED_OPCODE` prefix plus a static opcode, so it costs two indirect dispatches where an ordinary opcode costs one — the interpreter's own computed-goto jump into the prefix handler, and then a second jump through a 103-entry table shared by every static opcode. Disassembling the built interpreter shows that second dispatch as a single jump site, inside a computed-goto scheme whose whole purpose is to give each opcode a dispatch site of its own. It is a property of the 3.14 port rather than of the design: the 3.12 interpreter in the same source tree has no `EXTENDED_OPCODE` at all and reaches `LOAD_LOCAL` or `PRIMITIVE_BINARY_OP` through the main dispatch table like any other instruction. The deeper reason, though, is that the interpreted path does not implement primitives as primitives at all. In the 3.14 interpreter an `int64` lives on the operand stack and in the frame as an ordinary `PyLong`: `PRIMITIVE_BINARY_OP` takes two `PyObject*`, unboxes them, and allocates a fresh `PyLong` for the result; `STORE_LOCAL` re-resolves the declared primitive type through the class loader on every execution and boxes the value again; `LOAD_LOCAL` obtains its slot index by converting a Python integer out of `co_consts`. The typed form therefore pays the allocation it exists to remove and adds a type resolution and a second dispatch on top — which is also why the adaptive path is worth 43 % here, since its caches are what remove the per-store type resolution. Measured directly, one primitive `s = s + i` on `int64` costs 111 ns against 28 ns for the same statement on ordinary Python integers on the same interpreter. The only consumer that keeps an `int64` in a register is the JIT, and that is the practical reading: primitive opcodes are a notation for a compiler to consume. Interpreted, they do the same boxing as ordinary Python integers, so they cost four times what the code they replace costs.

The JIT's own compilation latency is worth recording next to Numba's: CinderX compiled the eight kernel functions in 1.1–5.2 ms each, i.e. 41–194× cheaper than Numba's 215 ms for one signature (§4.2). A runtime JIT that pays a few milliseconds per function has effectively no break-even point — which makes its steady-state behaviour on untyped code, rather than its warm-up, the thing that decides for or against it.

Result: what decides the cost of a collection is not the thread count. The parallel collector is the one CinderX feature whose own setting turns out to be the wrong variable, and the source says why. Only `deduce_unreachable` — reference subtraction and reachability marking — runs on the worker threads; finalisers and freeing stay on one, so whatever share of a collection those two phases are is the share the option can address. And an idle marking worker does not sleep — `Ci_gc_steal_backoff` spins — so spare threads become contention as soon as there is a live set to mark.

What does move the number is what the collector can see, which is what a fork-based server changes on purpose. Figure 8 (bottom right) measures one worker's shape: a long-lived state of $8 \cdot 10^5$ objects plus $8 \cdot 10^5$ objects of per-request garbage, collected with the state visible, after `gc.freeze()`, and after CinderX's `immortalize_heap()` — which runs a full collection, calls `freeze` and then immortalises the survivors so their refcounts stop dirtying pages after a fork. With the state visible a collection takes 187 ms and the parallel collector makes it 276 ms (1.48× slower, $t = -139.2$). Once the state is out of the collector's view a collection takes 129 ms — 1.45× faster than leaving it in ($t = 161.9$) — and only then is the parallel option positive at all, at 1.06× ($t = 13.7$).

Two things follow. The lever on collection cost is heap composition, and it is worth 1.45× where the thread count is worth 1.06× in the one configuration that does not make things worse. And the CinderX runtime does not change collection cost by itself: stock CPython 3.14.6 collects the same heap in 186 ms against CinderX's 187 ms, a difference the significance test declines to resolve (1.004×, $t = -1.33$). Whatever this runtime is worth, it is not that its collector is faster.

Result: what it cost to get there. CinderX built and ran natively on this host in both configurations, with zero compiler errors in each: all eight kernel functions force-compiled and produced correct results, and Static Python ran end to end in both. What the build needed is in the build scripts and logs in the artifact, which are the source for this paragraph and are not measurements: clang is mandatory and is used for the fork, the stock reference and the extension alike, each configuration needs its own source tree, and the Static Python loader silently produces boxed bytecode unless `cinderx.compiler.strict.loader.install()` is called — a call documented only in the Static Python tutorial, not in any API reference, and the difference between measuring Static Python and measuring ordinary bytecode under a different name.

Three traps sit in the measurement rather than in the build, and each of them produces a number rather than an error. Compiling a kernel before its first call — the obvious thing to write, and what `force_compile` invites — compiles unspecialised bytecode: on the same six kernels that path gives 0.85–1.53× stock where the JIT's own threshold gives 0.33–0.79, so it reverses the sign of the result. Calling `force_compile` on a function that has already run produces compiled code that is never entered, and `is_jit_compiled` answers True while every call goes through the interpreter. And `is_jit_compiled` is not a reliable witness in the other direction either: with the HIR inliner on, which is the default, it answers False for a function the JIT is running. `count_interpreted_calls` is the only one of the three that says what is executing, which is why the suite gates on it. A fourth belongs with them and is not the JIT's fault: `pyperf` scrubs the environment of its worker processes, which is how a benchmark stops depending on the shell it was started from, and the JIT reads its configuration from the environment — so a configuration set for a run reaches the master and not the workers, and the workers are what gets timed. Our first pass at the option sweep above measured the default nine times, successfully and without a warning. The suite now states the intended setting on the command line, which `pyperf` does rebuild for workers, and aborts in any process where it did not arrive.

One option had to be repaired before it could be measured at all: `-X cinderx-jit-stable-frame=0`, the conservative setting for code the stability assumption does not hold for, segfaulted on a nine-line script. The two-function fix is in the artifact, and with it CinderX's own static test suite runs to completion in that configuration. The option also silently disables the HIR inliner, which its help string does not mention.

Finding. "Experimental" does not describe this project usefully, and Table 7 replaces it with a per-capability account. Cinder/CinderX is not unstable: it builds in both configurations with zero compiler errors, it runs, and its JIT is correct. It is not free, however, and it is easy to measure backwards. On our six untyped kernels its runtime costs 1–33% in either configuration — the frame evaluator it installs replaces the loop CPython specialises — and its JIT then more than repays that, running them at 0.33–0.79× stock, between 1.3× and 3.1× faster. Of the three runtime features we could exercise, one is a large win on the axis it addresses (lazy imports), one does nothing on our probes (lightweight frames, within 3 % on the bare fork and 13–37% dearer under the CinderX runtime), and one turns on a variable that is not its own setting (the parallel collector, which is 1.48× slower while the long-lived heap is visible to it and 1.06× faster once immortalisation has taken it out of view). Static Python adds 11.4× on top of the JIT on a typed kernel, or 7.5× on the slow layout mode, and asks for a rewrite in primitive types and a boot sequence documented only in a tutorial — and those primitives are worth having only because something compiles them; interpreted,

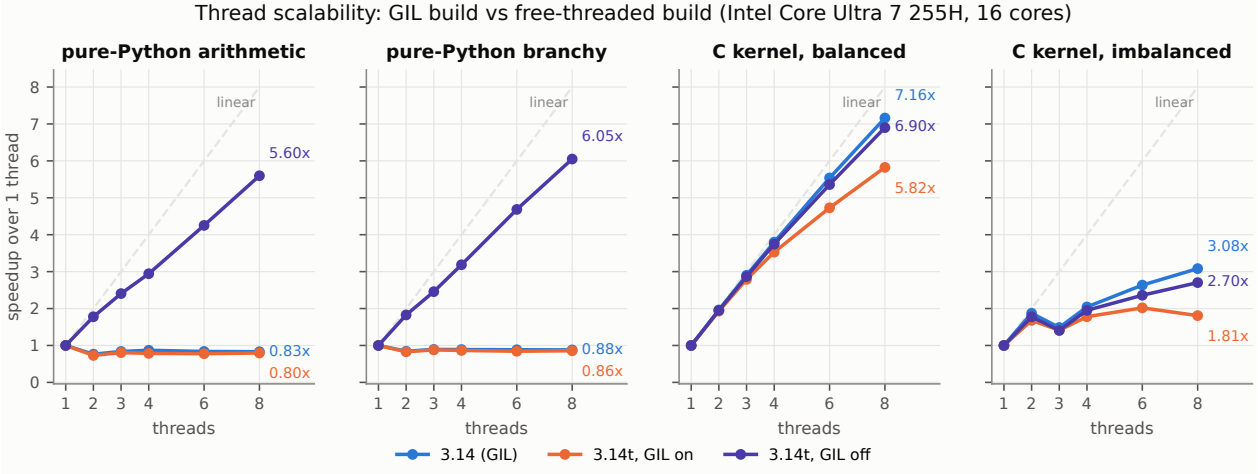

Figure 10: RQ5: thread scalability. Speedup is relative to the same configuration at one thread; the dashed line is linear scaling.

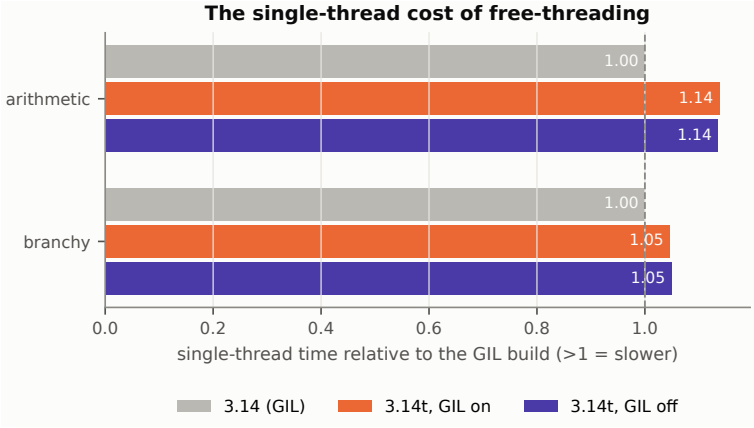

Figure 11: RQ5: what free-threading costs when only one thread runs.

even with the specialising path we had to complete, they are 5.8× slower than the boxed code they replace. The entry the project earns is therefore wider than the one we first measured and still precise: the JIT is worth having on ordinary code provided it is allowed to compile after the interpreter has specialised, lazy imports pay for a service that imports far more than it executes — 0 of 200 module bodies here, and nothing at all where the bodies do run — and the order-of-magnitude figure belongs to teams able to rewrite hot code with typed primitives and maintain a fork. What our measurements do *not* cover, and what an evaluation of the project should, is the workload it was built for — long-lived server code, where the frame machinery we saw no benefit from is supposed to matter, and where the heap composition that decides the collector's result is arranged deliberately rather than, as here, constructed to measure it.

8 RQ5: how far does the free-threaded build scale?

Question. The free-threaded build [9] removes the interpreter lock. How far does it scale on CPU-bound Python code, and what does it cost when only one thread runs?

Setup. Three configurations of the same 3.14.6 source: the default build (GIL), the free-threaded build with `PYTHON_GIL=1` (GIL re-enabled — this isolates the cost of the free-threaded build itself from the effect of removing the lock), and the free-threaded build as intended. Three workloads: pure-Python arithmetic, pure-Python branchy byte classification, and a C kernel called through `ctypes` (which releases the GIL, so it is expected to scale in every configuration). Total work is fixed and split across $T \in \{1, 2, 3, 4, 6, 8\}$ threads; a process-pool point is measured for reference. This suite is the one exception to the paper's affinity policy: every other suite is pinned to the six performance cores, and a thread sweep pinned there would cap at six and manufacture a plateau, so it runs on CPUs 0–13, the six performance and eight efficiency cores.

Result: the lock is what stops scaling, and removing it delivers. The two interpreters involved are built from the same 3.14.6 tarball by the same script with the same compiler, and their `CONFIG_ARGS` differ in exactly one token, `-disable-gil`. The third configuration is the free-threaded build again with `PYTHON_GIL=1`, which puts the lock back without changing the binary. The comparison therefore isolates the lock, then the build, and never the packaging.

Figure 10. All three configurations were measured back to back, and their machine probes agree to within 0.25% (4.069, 4.059 and 4.062 ms), so the comparison between them is not standing on drift; the single-thread repeat control moved by at most 1.03 %.

Pure-Python arithmetic does not scale under the GIL — it gets *worse*, 0.76× at two threads and 0.83× at eight — and it behaves the same way on the free-threaded build when the GIL is switched back on with `PYTHON_GIL=1` (0.73× and 0.80×), which is what makes the lock rather than the build responsible. With the lock off, the same workload reaches 1.78× at 2 threads, 2.94× at 4, 4.25× at 6 and 5.60× at 8; the branchy workload reaches 1.83×, 3.19×, 4.69× and 6.05×. Scaling is real and sublinear from the first step: per-thread efficiency on the arithmetic workload is 0.89 at two threads, 0.74 at four and 0.70 at eight, and the eighth thread is on an efficiency core — the sweep has six performance cores available and takes what is left from the efficiency class.

The native-code control makes the mechanism explicit. A balanced C kernel called through `ctypes` scales in *every* configuration — 7.16× under the GIL, 5.82× on the free-threaded build with the GIL on, 6.90× with it off — because `ctypes` releases the lock for the duration of the call. If the hot loop is already in C, removing the GIL buys nothing; it is bytecode execution that the lock serialises. The deliberately imbalanced variant of the same kernel reaches only 1.81–3.08× at eight threads regardless of configuration, and is not even monotonic in the thread count — all three configurations are slower at three threads than at two, and the free-threaded build with the GIL on is at its best at six threads (2.02×) and falls back to 1.81× at eight — a reminder that a scaling curve can be flattened by work partitioning alone, and that a reported "N× on threads" says as much about the partitioning as about the runtime.

For reference, the same arithmetic across eight *processes* on the ordinary build gives 5.62× — on the same baseline as Figure 10, the faster of that configuration's two single-thread measurements — with the pool already warm so that only submit, compute and collect are counted. That number and free-threading's 5.60× are not directly comparable, because each is relative to its own build's single thread and the free-threaded build starts 11 % behind (355.7 against 320.3 ms on this sweep's own single-thread baseline; the next paragraph measures the same tax as 1.14× on a different, much shorter workload). In wall time the eight processes finish in 57.0 ms against the eight free-threaded threads' 63.6 ms, i.e. 1.115× *faster* ($t = -34.3$, significant), not level with it. The control that separates the build from the mechanism is in the same suite: eight processes on the *free-threaded* build take 64.4 ms, so against its own eight threads at 63.6 ms the threads win by 1.3 % ($t = 2.67$, significant). Processes do not beat threads here — the free-threaded build's single-thread tax does, and it is charged to both of its own curves. What free-threading avoids is the address-space duplication and the serialisation of arguments and results; threads never pay either, and for processes this workload — four integers in, one integer out, with the pool already warm — is the cheapest version of them there is.

What free-threading costs. Figure 11. Single-threaded, the free-threaded build is 1.14× slower on arithmetic (14.26 against 16.20 ms) and 1.05× slower on the branchy kernel (25.23 against 26.47 ms). Re-enabling the GIL on the same binary does *not* recover it — 1.14× and 1.05×, the same two figures to two decimals — which places the cost in the build rather than in the lock's absence: the free-threaded interpreter pays for its different object layout and reference-counting scheme whether or not the lock is there. On the realistic mixed pipeline of §10 the same tax is 0.4 %, so these two microbenchmarks are the pessimistic end of it, and the second thread repays it either way.

Finding. Free-threading delivers, and it delivers about what the cores allow: 5.60× on arithmetic and 6.05× on branch-heavy code at eight threads, where the ordinary build delivers 0.83× and 0.88× — that is, on the ordinary build adding threads to this workload makes it slower. The `PYTHON_GIL=1` control on the same binary is what makes that attributable to the lock rather than to the build. Three entries belong next to it. The first is the price: 1.05–1.14× of single-threaded throughput on these microbenchmarks and 0.4 % on the mixed pipeline, and it is the build's price, not the lock's — the same binary with the lock switched back on pays it too — so it is charged to every part of a program, including the parts that never start a thread; the second thread repays it. The second is a precondition: if the hot loop is already native there is nothing to gain, because `ctypes` releases the lock and the same kernel scales 7.16× on the ordinary build. The third is that eight processes on the ordinary build finish this workload in 57.0 ms against the eight free-threaded threads' 63.6 ms — but that compares two interpreters rather than two mechanisms: within the same free-threaded

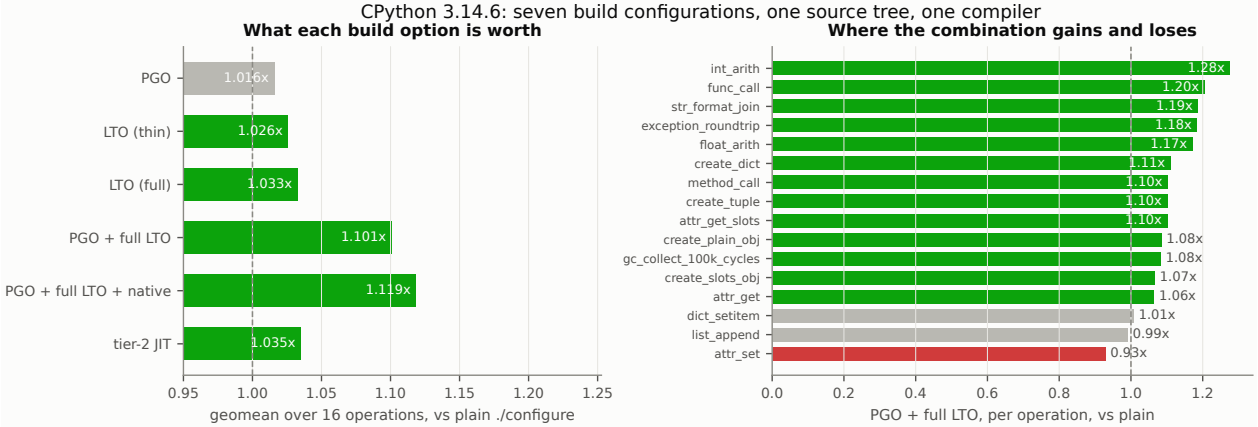

Figure 12: RQ6: seven build configurations of one CPython 3.14.6 source tree with one compiler. Left: geometric mean over the operation suite. Right: where PGO with full LTO gains and where it loses.

build, threads (63.6 ms) beat processes (64.4 ms), and the whole gap is that build's single-thread cost. The task also passes four integers in and one out, so the marshalling that threads never do at all is cheaper here for processes than on any other workload.

9 RQ6: how much is there in PGO, LTO and `-march=native`?

Question. The interpreter is itself a C program, so profile-guided optimisation, link-time optimisation and architecture-specific flags all apply to it. With the source, the compiler and everything else held identical, how much does each of them give, and do they compose?

Setup. Seven configurations, all from the one CPython 3.14.6 source tree vendored in the artifact, all compiled by the same `/usr/bin/clang`, differing only in the `configure` line: plain `./configure`; `-enable-optimizations` (PGO); `-with-lto=thin`; `-with-lto=full`; PGO with full LTO; the same with `CFLAGS=-march=native`; and `-enable-experimental-jit` (§9.1). The distinction between `thin` and `full` is worth stating because it is easy to lose: CPython's option is `-with-lto=[full|thin|no|yes]` and a bare `-with-lto` resolves to `-flto=thin` on this platform, so "PGO + LTO" as usually written means PGO with *thin* LTO. Each configuration runs the runtime operation suite of §6 under `pyperf`, and the numbers below are geometric means over its 16 operations relative to the plain build.

Result: the useful entry is the combination, not any single option. Figure 12, left panel, geometric mean over 16 operations, relative to a plain `./configure` build of the same tree:

- **PGO alone:** 1.016× — the smallest of the single options, and not uniform: it is worth 1.131× on string formatting and 1.111× on integer arithmetic while costing 5 % on attribute reads (0.946×, $t = -12.5$, significant).
- **LTO alone:** 1.026× **thin,** 1.033× **full** — and, unlike PGO, both are positive on almost every operation. The two are not the same build: full LTO is ahead of thin on integer arithmetic (1.0114×, $t = 3.35$), on method calls (1.0347×, $t = 13.8$) and on string formatting (1.0254×, $t = 9.65$), all significant. The margin is one to three and a half per cent, so thin LTO recovers most of what full LTO is worth at a much lower link cost.
- **PGO with full LTO:** 1.101× — more than the two options multiplied together ($1.016 \times 1.033 = 1.050$), so this is where the build-level row actually lives. It is also broad: fourteen of the sixteen operations improve, led by integer arithmetic 1.276×, function calls 1.204×, string formatting 1.187×, exception round-trips 1.184× and float arithmetic 1.172×. Only attribute writes get worse (0.928×); `list.append` (0.990×, $t = -3.86$) and `dict` assignment (1.007×, $t = 4.21$) move by under 1%, both differences resolved by the test.
- **Adding `-march=native` on top:** 1.119× — a further 1.6 %, and the best configuration measured. It is not free of the same trade: attribute writes fall further, to 0.902×.

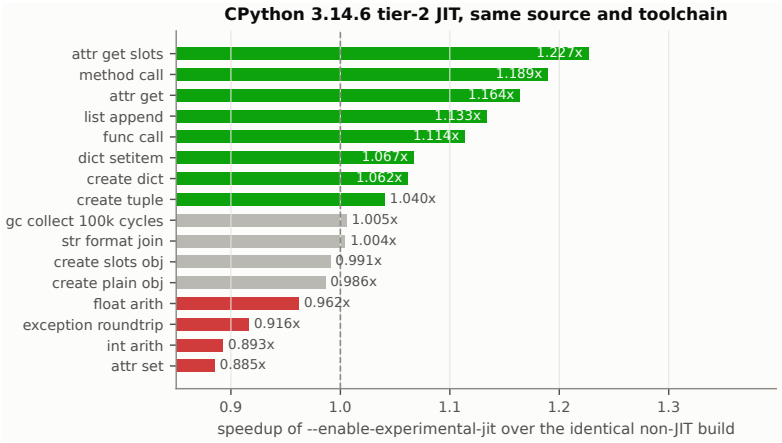

Figure 13: RQ6b: `-enable-experimental-jit` against the identical build without it, per operation. Same source, same compiler, same suite.

The compounding is the substantive result of this section, and it is the opposite of what the per-option numbers suggest: PGO on its own is worth 1.6 %, full LTO on its own 3.3 %, and the two together 10.1 %. A recommendation assembled from single-option measurements would have ranked this row last; measured as a build configuration rather than as a list of flags, it is worth more than a release of interpreter work (§6). The plausible mechanism is that a profile lets whole-program inlining choose usefully across the evaluation loop instead of inlining by size heuristics — a choice neither option can make alone. We report the effect, and the per-operation profile that makes it visible, without identifying the pass responsible. The one operation that resists it in every configuration — attribute writes, 0.88–0.98× in all six — is the one to check on any program that is dominated by them.

Start-up latency moves in the same direction here, and by less: the stdlib-import start falls from 69.4 ms on the plain build to 66.2 ms with PGO and 65.6 ms with PGO and full LTO, with or without `-march=native`. A configuration that pays off in steady state is therefore not paying for it at start-up on this build — but start-up is measured as whole-process wall time for each of the same seven builds precisely because that is not something to assume.

Finding. The build-level row is real, it is bigger than it looks option by option, and it is not the row people think it is. PGO alone is worth 1.6 % and LTO alone 2.6–3.3 %; the recommendation that pays is the combination, at 10.1 %, rising to 11.9 % with `-march=native`, which is the one place in this paper where the architecture flag is a consistent (if small) win. Against the 3.2–42.8× available one row up (an alternative runtime, §11) or the 32–137× two rows up (compiled kernels, §4), a 12 % build-level gain is still a last-mile optimisation — worth taking when the interpreter is yours to build, never worth prioritising — but it is now worth taking as a whole rather than as a list of options, and worth measuring on attribute-heavy code before shipping.

9.1 RQ6b: does CPython's own experimental JIT help yet?

Setup. CPython's own tier-2 JIT [11], experimental since 3.13, is routinely listed among the available options and rarely measured. Its build is the seventh configuration of §9: the same source and the same compiler as `plain`, with `-enable-experimental-jit` as the only difference. Building it additionally requires LLVM 19 for the stencil generator, which is a documented build requirement of the feature and the only reason a second toolchain appears anywhere in this artifact.

Result: the tier-2 JIT improves more operations than it slows. Figure 13. The JIT-enabled build is a geometric mean of 1.035× the plain build — about 3.5% faster. Ten of the sixteen operations improve, and the improvements land where a tier-2 trace optimiser is supposed to help: attribute reads on a slotted instance 1.227×, bound-method calls 1.189×, attribute reads 1.164×, `list.append` 1.133×, function calls 1.114×, `dict` assignment 1.067×. Six get worse: attribute writes at 0.885× are the largest of those, and the one hardest to explain away is next — integer arithmetic, the most JIT-friendly case in the suite, at 0.893×. Exception round-trips are 0.916× and float arithmetic 0.962×; instance creation moves by under 1.5 % either way (0.986× plain objects and 0.991× slotted), both differences resolved by the test. Start-up is unchanged (69.4 ms for the stdlib import, against 69.4 ms for the plain build), so the JIT is not being paid

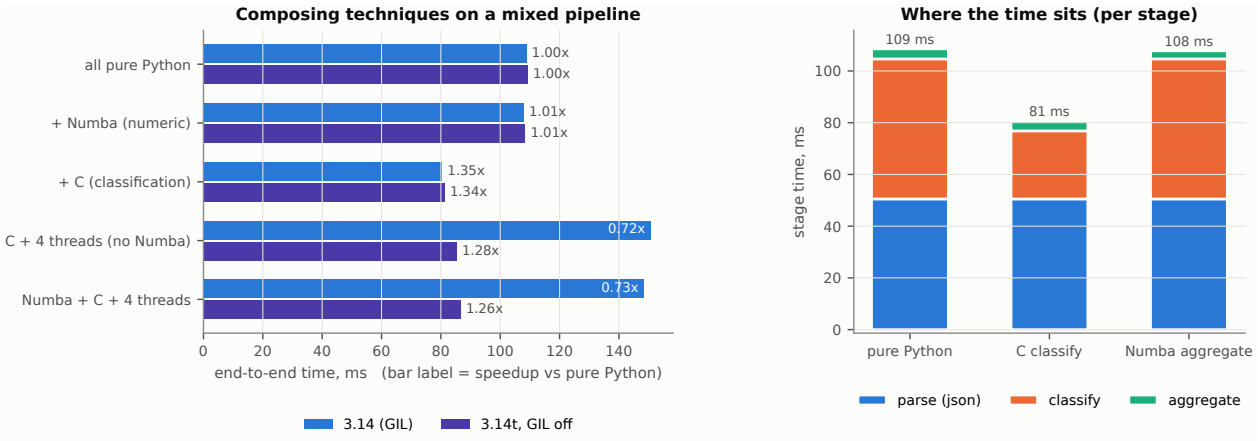

Figure 14: RQ7: stacking techniques on the mixed pipeline (left) and where the time sits per stage (right).

for at process start on this suite. The build is genuine — `_Py_JIT` and `_Py_TIER2` appear in `PY_CORE_CFLAGS`, and `sys._jit.is_available()` and `sys._jit.is_enabled()` both report true — so this is the JIT running, not a JIT that failed to engage.

Finding. A qualified "yes" with a number attached, and a caveat inside it. On this host, at this version, CPython's own JIT is worth about 3.5% on interpreter-level operations — the same order of magnitude as the build-level row it competes with, and available by flipping one `configure` switch rather than by running a PGO training pass. The caveat is the distribution rather than the mean: it pays on attribute and call paths by up to 23 % and loses 11 % on integer arithmetic, so which side of the mean a program lands on depends on what it does. It is still correctly labelled experimental and our suite is microbenchmark-shaped rather than trace-shaped — but the row is no longer empty.

10 RQ7: do the techniques compose?

Question. If no single tool wins everywhere, the practical answer is a composition: a JIT for the numeric part, native code for the irregular part, an optimised runtime underneath, threads for scale. Does such a composition add up on a realistic mixed pipeline?

Setup. An ordinary mixed pipeline over 20 000 JSON records: parse with the stdlib `json` module, classify a string field per record (branchy), fold the numeric fields (compute-bound; the *aggregate* stage in Figure 14), emit a summary. Four variants add one technique at a time, and each must produce a byte-identical summary. The whole set runs twice, once on the ordinary 3.14.6 build and once on the free-threaded one.

Result: the stage profile decides which technique is worth applying. Figure 14. The pure-Python pipeline takes 108.9 ms on the ordinary build, split as 50.5 ms parsing, 54.1 ms classification and 3.9 ms numeric fold. Stacking the techniques one at a time gives:

- **Numba on the numeric stage: 1.01× end to end.** The stage itself improves from 3.87 to 3.10 ms, but it is 3.6% of the pipeline, so the gain disappears — Amdahl's law, measured. The stage was worth 3.6% of the run before the technique was applied to it, and that number was available beforehand.
- **C on the classification stage: 1.35× end to end.** The stage improves 2.06× (54.1 → 26.3 ms) and it is 50% of the pipeline. Note that this includes 20 000 `ctypes` crossings at 166 ns (≈ 3.3 ms, §5).
- **Four threads on the GIL build: 0.72× — a regression.** Both threaded variants are *slower* than the sequential one (150.7 and 148.5 ms against 108.9 ms). The per-record loop is Python-level bytecode, so the GIL serialises it, and the chunking and thread management are pure overhead.
- **Four threads on the free-threaded build: 1.28× — a real gain that is still not enough.** The same code drops from 150.7 to 85.4 ms, i.e. removing the lock turns a 38 % regression into a 28 % improvement. But the sequential Numba-plus-C variant is faster still, on both builds (80.5 ms with the GIL, 81.3 ms without).

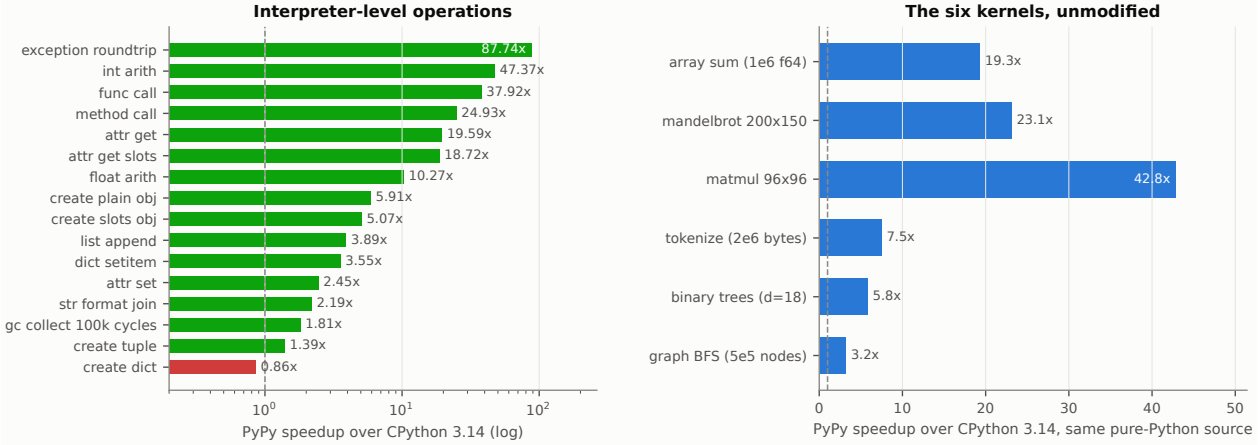

Figure 15: RQ8: PyPy 3.11 against CPython 3.14.6. Left: the sixteen operations of the runtime suite of §6, which PyPy runs unchanged. Right: the six kernels, with the pure-Python sources unmodified.

The fully-stacked variant — Numba, C and four threads — now runs on both builds, because Numba installs and imports on the free-threaded interpreter. On the GIL build it is 0.73×; on the free-threaded build 1.26×, which is below what two of its three techniques achieve without the third. The best configuration measured anywhere in this section is Numba plus C, sequential, on the ordinary build: 80.5 ms, 1.35×. The free-threaded build starts this pipeline only 1.0035× behind the ordinary one, so its penalty is not what decides the comparison; the partitioning is.

Finding. Composition is the right principle — the best configuration we measured does combine two techniques, at 1.35× over the pure-Python baseline — but it is not additive, and a recommendation has to say so. On this pipeline two of the four techniques contributed nothing (1.01×) or less than nothing (0.72× on the ordinary build), and which ones failed was decided by the stage profile rather than by the technology: threads have to divide a stage that is not already the fastest thing in the pipeline, and here the stage they were given had already been moved to C. The operational form of the rule is therefore: profile, apply the technique that fits the dominant stage, then *re-profile* — because once classification moved to C, parsing became 50.5 ms of the remaining 80.5 ms, and neither a JIT nor threads can touch it.

11 RQ8: where does PyPy stand on the same kernels?

Question. Every technology in RQ1 and RQ2 asks for a code change. PyPy asks for none: it is a tracing JIT for ordinary Python bytecode [4]. What does it deliver on the same kernels with the source untouched, and what does it cost?

Setup. PyPy 3.11 (v7.3.20, implementing Python 3.11.13) runs our kernels and the runtime suite unchanged on the same host, under `pyperf`, exactly as every CPython build does. NumPy, Cython, pybind11, `ctypes` and the Rust cdylib all work there and are measured. Numba is the single exception: it requires `llvmlite`, for which no PyPy wheel is published and which does not build against PyPy, so the suite records its rows as `unavailable` with a `ModuleNotFoundError` rather than as a slow result.

Result: a wide distribution around a large geometric mean. On the operation suite of §6, run on PyPy without changes (Figure 15, left), PyPy is a geometric mean of 7.3× faster than CPython 3.14, and the distribution is extremely wide: exception round-trips 87.7×, integer arithmetic 47.4×, function calls 37.9×, bound-method calls 24.9×, attribute reads 19.6× — and, at the other end, building a dict literal 0.86×, where PyPy is the slower of the two, tuple creation 1.39×, a cyclic garbage collection 1.81× and string formatting 2.19×. On the six kernels, with sources unchanged (Figure 15, right), PyPy returns 42.8× (matmul), 23.1× (mandelbrot), 19.3× (array sum), 7.5× (tokenize), 5.8× (binary trees) and 3.2× (BFS).

Three consequences follow.

First, **on the compute-bound class PyPy is in the same league as the compiled implementations, at zero code cost**. Cython, Numba, C and Rust reached 32–137× on these kernels (§4) — but each required either type annotations, a second language, or a build step. PyPy reached 19.3–42.8× on the same source files with no edit at all: 19.3× against the compiled band of 32× on the array sum, and 42.8×

against 72–137× on the matrix product. That is the same order of magnitude for none of the work, and for a codebase that is not allowed to grow a compiled component it is the cheapest large factor available in this entire study.

Second, **the compiled ecosystem is not unavailable on PyPy — but the C-API-heavy paths through it cost an order of magnitude**. NumPy, Cython and pybind11 all import and run. Cython and pybind11 land within 4 % of their CPython times on all three compute kernels, and NumPy on two of the three (Cython's array sum 1.025 against 1.011 ms, the pybind11 mandelbrot 0.774 against 0.752 ms, NumPy's matrix product 0.051 against 0.049 ms), and Cython is actually *faster* on PyPy than on CPython for two of the branchy kernels (1.04× on tokenize, 1.32× on BFS). Then there are two collapses, and both have the same shape. Cython's binary-trees kernel takes 493.8 ms on PyPy against 25.9 ms on CPython — 0.05× — and it is the one kernel whose `cdef class` allocates $2.6 \cdot 10^5$ CPython objects per call, every one of which PyPy has to emulate. NumPy's mandelbrot takes 77.1 ms against 9.98 ms — 0.13× — and it is the one NumPy kernel that is a long chain of small masked passes rather than a single dispatch into compiled code.

Third, **the same rule locates what happens to the FFI**. Running our C and Rust kernels through `ctypes` on PyPy costs nothing on five of the six kernels (0.96–1.07× of CPython, in both directions), and is catastrophic on the sixth: tokenize, which passes a 2 MB `bytes` object per call, takes 118.5 ms through C and 118.5 ms through Rust on PyPy against 22.3 and 22.4 ms on CPython — 0.19× for both. Where nothing but integers and preallocated buffers crosses, the penalty disappears entirely. What the six kernels separate is therefore *what* crosses the boundary rather than how often it is crossed: the cost is in marshalling the argument, not in making the call.

What it costs. PyPy starts more slowly and holds more memory: a bare interpreter start is 26.1 ms against CPython's 10.6, importing the standard set of modules 95.4 against 66.3 ms, and maximum RSS after the same suite 90.5 MB against 29.8. For a long-running service none of that is visible; for a short-lived process it is most of the run. One cost does not show up on this host at all. At the time of measurement PyPy implements 3.11, so a codebase on a later language version cannot use it; and NumPy removed PyPy support in February 2026 [5], so the NumPy-on-PyPy rows above are tied to the 2.4 series and do not carry forward. Both are checkable facts about the version a reader would install, and a recommendation that has to pay off over a long horizon has to weigh them alongside the start-up cost.

Finding. An option easily left out of the comparison: for pure-Python compute-bound and call-heavy code that must stay pure Python, an alternative runtime is worth 3.2–42.8× without touching the source — the largest factor we measured that asks for no edit at all. It does come with a constraint, but a narrower one than "PyPy and native code do not mix". Nearly all of the compiled ecosystem works there and costs a few per cent; what costs an order of magnitude is traffic across the C-API boundary — a 2 MB buffer marshalled per `ctypes` call, or a quarter of a million emulated CPython objects per call inside a Cython extension. So the two recommendations "switch to PyPy" and "move the hot loop behind an FFI" can be stacked, provided the boundary is crossed rarely and cheaply; a per-record call with a large payload is the case that makes them exclusive. Numba is the one technology with no path at all: it has no PyPy build.

12 Discussion: a decision table instead of a ranking

Advice in this area is usually organised by technology — "use Numba for this, use C for that". The measurements put the workload first and the technology second, for three reasons that recur across all eight questions. The index that works is what the workload does, with the technology and its price in the cells.

1. On compute-bound work the technologies are interchangeable; where allocation dominates they are not. On compute-bound scalar loops, Cython, Numba, C behind two different FFIs, and Rust agree to within 0.2% on the reduction, and where they do not agree they differ by 18% on one kernel and 1.8× on another, in both directions with no consistent winner — against the 32–128× that compiling at all is worth (§4). The measurements therefore do not decide among them; what would — build complexity, debuggability, code ownership — lies outside what this study measures. What *does* distinguish outcomes is which class the workload belongs to: the same four technologies span 52–61× on the compute-bound kernels and 7.3–13.6× on the branchy and allocating ones (§5) — and inside that second class they stop being interchangeable too, separating by 3.7–5.3× exactly where small objects are allocated.

2. The decisions that dominate are about what the program may do, not about which tool does it. Choosing to write the hot loop in a language that can be compiled at all is worth more than any

choice made within that language — except where the object model is the bound, and there the two are the same size: compiling the binary trees at all buys Cython 4.9×, and choosing an implementation that owns its own memory buys 3.5–5.3× on top of it (§5) — and flattening the data structure without compiling the loop made our graph traversal 12% *slower*. Deferring module execution is worth four orders of magnitude of import latency (§7) and changes when import errors surface. Allowing floating-point reassociation is worth 2.42× on a reduction, and it changes results — and it needs no compiler flag at all, since regrouping the accumulation in the source is worth 2.74× at unchanged flags (§4.1). Each is a decision about what the program is allowed to do, and each dominates the tool choice made under it. The flags below those decisions are not a substitute for them: moving from `-O2` to `-O3` was worth nothing measurable on two of our three kernels and 0.1% on the third, while `-march=native`, the one flag that did change the emitted code everywhere, made one kernel 1.25× faster and another 1.80× *slower* (§4.1).

3. Every gain has a cost, and the cost is the part that is usually not reported. JIT compilation costs 215 ms of latency in the one case we measured, which is 7 calls or 7335 calls of work depending on input size (§4.2). Free-threading costs 1.05–1.14× of single-thread throughput (§8). Static Python costs a rewrite in primitive types and is 10.2× *slower* if the JIT is not also enabled (§7). An FFI costs 59–166 ns per crossing, against 31 ns for a pure-Python call: free for a 10 μs kernel, a real cost per record (§5).

Table 8 is the result: the workload classes of §1 with a measured effect and a price in every row. It is the form we would put in front of an engineer.

Four questions to ask of a performance figure. Working through the eight research questions of this paper produced a short checklist. What was the input size — a break-even point moves by three orders of magnitude across ours (§4.2)? Was the baseline the same program — a vendor BLAS against a naive triple loop is not a language comparison (§4)? Was one variable changed, or two — an interpreter's build options are worth several releases of interpreter work, so a version sweep on differently-built binaries measures neither (§6, §9)? And what did it cost — on the code it does not accelerate, at the boundary it introduces, in the semantics it relaxes (§8, §5, §4.1)? A figure that survives those four is worth acting on.

13 Threats to validity

One machine, and what that does not license. Every measurement here comes from one x86-64 laptop. Absolute times are properties of that machine; the ratios are what travel, and even they travel only as far as the mechanism behind them does. The machine-code evidence of §4.1 is the clearest case: which flags emit identical code is a property of this compiler and this target, and we can show that directly rather than warn about it in the abstract. So the finding to carry away is not a recommendation about a flag; it is that what a flag does to the emitted code has to be re-established per compiler and per target, and that the disassembly settles the question in seconds.

Core classes, and where pinning stops helping. Pinning the single-threaded suites to the performance class (§3) removes the largest source of unexplained dispersion but not all of it: nothing is *excluded* from those cores, so the operating system and any other work on the machine may still be scheduled there. Real isolation would need the cores reserved at boot, which we did not do. The thread-scaling study of §8 cannot be confined to the performance class — it sweeps up to eight threads and that class has six cores — so it runs on CPUs 0–13: the six performance cores and eight of the ten efficiency cores of Table 1. Its curve is reported whole rather than reduced to a single "$N\times$", because past six threads the added threads are on slower cores.

Kernel selection, and what six kernels cannot stand for. Six kernels cannot represent all Python workloads. They were chosen to instantiate the workload classes of §1, and the compute/branchy split is deliberately extreme so that the technologies' behaviour separates.

What the machine probe cannot see. The machine probe of §3 is single-threaded, and this study found the limit the hard way: during one pass, other work on the machine slowed a memory-bound NumPy benchmark by 13–34% while the probe reported the host as unmoved to within 0.03%. It was not wrong — contention on the cores a single-threaded loop does not use is invisible to it by construction. Benchmarks that are parallel or bandwidth-bound therefore carry a residual exposure that the probe does not cover, and the affected suite was re-measured on an idle machine rather than reported as it stood.

Machine steadiness, and what is still exposed. The machine probe shows this host moving very little: across the 46 result files that carry it, the slowest invocation is 1.014× the fastest and the ninetieth percentile is 1.009×. That is a measured statement rather than an assumption, and it is what the probe is for. Comparisons *within* one suite invocation are the least exposed: the suite is minutes long and its benchmarks are adjacent in time. Comparisons *across* invocations are the most exposed, and the version sweep is exactly that shape — six separate runs of the same suite, launched in sequence, with the cross-version ratios read across them. The machine probe bounds that: it is the same benchmark in all six, so its spread across the six runs is a direct measurement of how much the host moved while they were collected, and it is reported with the sweep. Where a claim turns on less than that spread, it is reported with that spread beside it (§3); where it turns on a factor, the drift is not the binding uncertainty. What remains is the resolution stated in §3: adequate for the factors this paper's conclusions turn on, and not adequate for a few per cent, which is why the few claims that turn on a few per cent carry a significance verdict.

Interpreter provenance. Building every interpreter here (§6) is a restriction as well as the control it was chosen for: what the paper measures is what these sources do when built this way, and an interpreter obtained some other way is a different binary whose behaviour is not established here.

14 Reproducibility

The artifact accompanying this paper contains the measurement layer, all kernels in all languages, the interpreter build scripts, the raw `pyperf` result files and the figure and table generators. One command runs all of it, taking a bare machine to both PDFs and pinning each toolchain to the version measured here; the steps it performs are listed below, because the order is load-bearing and the reasons for it belong on the record:

```
bash bench/bootstrap.sh                    # everything below, in this order, from a bare
                                           # machine; --check, --deps, --build and --measure
                                           # stop it earlier
bash bench/tune_machine.sh apply           # performance governor, turbo off -- before the
                                           # host is described, so the description is of the
                                           # machine that will be measured
bash bench/host_topology.sh                # CPU classes -> affinity + results/host.json
OUT=<dir> bash bench/build_uniform.sh      # every interpreter: one compiler, one
                                           # configure line, 3.10-3.14 + free-threaded
UNIFORM=<dir> bash bench/setup_env.sh      # venvs on those interpreters; build_all.sh
                                           # builds into them, so this comes first
CODON_DIR=<dir> PYPY=<pypy>/bin/pypy3 \
  bash bench/build_all.sh                  # Cython, pybind11, Codon, C (6 flag sets +
                                           # the accumulator ladder), Rust
OUT=<dir> LLVM19=<llvm19>/bin \
  bash bench/b5_buildflags/build_real.sh   # the seven build configurations; the tier-2
                                           # JIT one needs LLVM 19
REAL=<dir> bash bench/setup_env.sh         # and again, for those seven: they are built
                                           # without pip, so this pass has to come after
                                           # build_real.sh
bash bench/b2_branchy/codegen_check.sh     # results/codegen/summary.json, for the
                                           # C-vs-Rust instruction-count table
UNIFORM=<dir> REAL=<dir> PYPY=<pypy>/bin/pypy3 bash bench/run_campaign.sh \
    compute branchy versions spec threads builds pipeline pypy
OUT=/path/cinder bash bench/b6_cinderx/build_cinderx.sh
CINDER=/path/cinder bash bench/b6_cinderx/run_b6_native.sh   # both CinderX configurations
CINDER=/path/cinder bash bench/b6_cinderx/run_jitopts.sh     # the JIT-option ablation
<venv>/bin/python bench/b6_cinderx/run_gc_scale.py --label ...  # the collector sweep
<venv>/bin/python bench/plots/check_overlaps.py   # layout gate; must exit 0
<venv>/bin/python bench/plots/make_figures.py
<venv>/bin/python bench/plots/make_tables.py
<venv>/bin/python bench/plots/summarize.py        # every number quoted in the text
```

Every figure and table is written by those generators from `results/pyperf/`, and every number quoted in the prose is derived from the same files rather than from a note taken while running: `summarize.py` prints the medians and ratios next to the records they come from, and the significance verdicts are `pyperf compare_to` over the two sets of values. A result file is pyperf's own format, so it can also be read with `pyperf stats` and `pyperf compare_to` without any of our code. Each benchmark carries its suite, kernel, implementation,

parameters and configuration label in its metadata, and each suite writes a sidecar holding the correctness verdicts, the variants that could not be built, the host description and the structural observations that are not timings — so that nothing pyperf did not measure can be mistaken for something it did.

15 Conclusion

This paper set out to measure the menu of Python acceleration techniques under conditions that let them be compared: eight questions, one measurement protocol, one interpreter provenance, one documented host, and code for every number. What comes out of it is a table indexed by workload rather than by tool, and the sections below are the entries that a reader should take away from it.

What the technology buys. Compiled code — by any of the routes tested — is one to two orders of magnitude faster than the CPython interpreter on compute-bound kernels (31–32× on an array sum, 52–62× on a mandelbrot grid, 46–128× on a naive matrix product). Which route you take matters far less than taking one: on the scalar reduction they agree to within 4.4%, and on the other two kernels they differ by 21% and by 2.8×, in both directions and without a consistent winner. Against a factor of thirty to a hundred, speed does not decide between Cython, Codon, Numba, C and Rust; what does — build complexity, debuggability, code ownership — this study does not measure. It is a real spread, not a tie, and a project whose whole cost is one kernel should measure its own. Removing the GIL delivers 5.60× on arithmetic and 6.05× on branch-heavy code across eight threads, and a `PYTHON_GIL=1` control on the same binary — which instead loses 14–20% at eight threads — confirms that the lock rather than the build is responsible.

Where the mechanisms are not the ones usually named. The largest levers in the compute study are the decision to compile at all and, where the operation already exists in a library, a vendor BLAS; the flags underneath those decisions behave less uniformly than their names suggest. Disassembling every flag variant settles what each one actually changes: `-O2` and `-O3` emit byte-identical code on two of the three kernels and differ by a single instruction on the third, with run times to match, so the step between them buys nothing here. Only `-O3 -march=native` emits different code everywhere, and it is the flag whose effect runs in both directions (§4.1). Turning the vectoriser off under that flag changed neither the emitted code nor the time on any kernel, and SIMD instructions appear only once fast-math is granted, so the architecture flag is not buying vectorisation here. The floating-point contract is worth 2.42× on the reduction and is not really a flag either: four independent accumulators written into the C source, at plain `-O2` with no fast-math anywhere, are 13% faster than `-ffast-math` itself, and eight accumulators are worse than four. The row that would read "use native flags" reads instead "decide what the arithmetic is allowed to do, know that you are deciding it, and disassemble the result on the target you ship". On irregular code, branching is not what defeats a compiler (Cython and C are within half a per cent on our graph traversal, and the native routes lead by at most 17% on a byte scanner) — *allocation* is, and that is where C and Rust pull ahead by 3.7–5.3×.

Where the answer is a distribution rather than a number. Interpreter progress from 3.10 to 3.14 is real and uneven: 3.11 delivered the step (attribute reads 1.84×, method calls 1.76× against 3.10), 3.12 gave part of it back — it is slower than 3.11 on 14 of our 16 operations, all 14 significant, although it is the release in which `None` became immortal — 3.13 changed little, and 3.14 recovered to 1.97× and 2.04×, while building a dict literal on 3.14 runs at 0.74× its 3.10 rate. At the build level the useful entry is the combination rather than any single option: PGO on its own is worth 1.02×, PGO with *full* LTO 1.10×, and adding `-march=native` to that 1.12×; attribute writes are the one operation that loses in all six configurations, at 0.88–0.98×. CinderX resists a one-word verdict in the same way. It builds in both of its configurations with zero compiler errors, its JIT force-compiled all eight functions we handed it without a failure, and every kernel passed the correctness gate. Its runtime then costs 1–33% on our six untyped kernels in either configuration, which puts that cost in the code path rather than in a build flag, and its JIT repays it on all six at 0.33–0.79× stock — but only when compilation follows specialisation rather than preceding it. Its lightweight frames moved our probes by under 3% on the bare fork. Its parallel collector moved its probe the wrong way while the worker's long-lived heap was visible to it (1.48× slower) and the right way only once `immortalize_heap()` had taken that heap out of view (1.06× faster), which makes heap composition the variable rather than the thread count. Its one large win (Static Python, 11.4× on top of the JIT) is reachable only through a boot sequence documented only in a tutorial and only for rewritten typed code. Measuring the adaptive configuration was itself work: the specialising path for static opcodes had been carried to 3.14 without the handlers it specialises into, so Static Python raised on its first specialisation until we supplied them — and once supplied, the flag takes 43% off interpreted Static Python and nothing at all off the compiled kind, which is the clearest statement in the study that primitive opcodes are an intermediate representation for a JIT rather than a faster way to interpret.

The costs, which are the part usually missing. Four numbers that decide a choice and are rarely reported: the point at which a JIT pays for itself, which moves from 7 calls to 7335 depending only on input

size, against one observed 215 ms of compile latency; the cost of an FFI boundary, 59–166 ns per crossing (§5); the single-thread price of free-threading, 1.05–1.14×, which the free-threaded *build* charges whether or not the lock is off and which the second thread repays; and the cost of Static Python, which is 10.2× *slower* than boxed Python if its JIT is not also enabled, and still 5.8× slower with the specialising interpreter path switched on. Two further results belong here because they are cost-dominated: PyPy returns 3.2–42.8× on unmodified pure-Python code — the largest factor here that asks for no edit — but the one kernel whose `ctypes` call hands it a two-megabyte buffer costs 5.3× more there than on CPython (118.5 against 22.3 ms) while the calls that pass only integers are within 7%, so "switch runtime" and "move the hot loop to C" can exclude each other; and CPython's own experimental JIT is a 1.04× proposition overall that is not free either — it improves 10 of our 16 operations and slows 6, integer arithmetic and attribute writes to 0.88–0.89×.

On method. Three of our own results were wrong before the protocol caught them (§3), which is the whole argument for having one. What caught them was cheap: delegating the timing to `pyperf`, alternating measurement where a claim turns on a few per cent, running the correctness gate in the process that will do the timing, and probing the host rather than the program.

Practically, composition survives as a principle with a sharper rule attached: on our mixed pipeline, Numba on a stage worth 3.6% of the run is worth 1.01×; moving the classification stage — half the run time — into C is worth 1.35×; and spreading the work over four threads on top of that gives 0.72× under the GIL and 1.26–1.28× on the free-threaded build, still below the 1.35× the one well-aimed change already delivers. Composition is not additive; it is profile-directed, and stacking past the profile takes performance back. Optimisation decisions should be indexed by what the workload does — loop over arrays, allocate objects, traverse a graph, start up, spread across cores — and not by which tool is fashionable, because within a workload class the spread between tools was small next to the decision to use one at all, while across classes the same tool differed by an order of magnitude. Table 8 collects those entries, and every number in it is reproducible from the accompanying artifact with two commands.

References

[1] V. Stinner. *pyperf: toolkit to write, run and analyze benchmarks.* Version 2.10.0. `https://github.com/psf/pyperf`.

[2] Python Software Foundation. *pyperformance: the Python benchmark suite.* `https://github.com/python/pyperformance`.

[3] Meta Platforms. *Cinder: Meta's performance-oriented fork of CPython, and CinderX, the extension package that carries its JIT, Static Python, parallel collector and lazy imports.* `https://github.com/facebookincubator/MetaPython` and `https://github.com/facebookincubator/cinderx`.

[4] The PyPy Team. *PyPy 3.11 (v7.3.20).* `https://pypy.org`.

[5] NumPy contributors. *MAINT: drop pypy* — pull request #30764, merged 3 February 2026. `https://github.com/numpy/numpy/pull/30764`.

[6] S. K. Lam, A. Pitrou and S. Seibert. *Numba: a LLVM-based Python JIT compiler.* LLVM-HPC '15.

[7] S. Behnel et al. *Cython: C-extensions for Python.* `https://cython.org`.

[8] A. Shajii, G. Ramirez, H. Smajlović et al. *Codon: A Compiler for High-Performance Pythonic Applications and DSLs.* CC '23.

[9] S. Gross. *PEP 703 — Making the global interpreter lock optional in CPython.* 2023.

[10] M. Shannon. *PEP 659 — Specializing adaptive interpreter.* 2021.

[11] B. Bucher and S. Ostrowski. *PEP 744 — JIT compilation.* 2024. Draft.

[12] E. Snow and E. Elizondo. *PEP 683 — Immortal objects, using a fixed refcount.* 2022.

[13] B. Cannon and D. Viehland. *PEP 523 — Adding a frame evaluation API to CPython.* 2016.

[14] The PyPy Team. *PyPy speed centre.* `https://speed.pypy.org`.

Table 8: Decision table from the measurements. Every factor is from the single measurement host of Table 1; "cost" lists what the change also brings.

workload	action that paid off	measured	cost / caveat
numeric loop over an array	any compiled path (Cython / Numba / C / Rust)	32×	a build step; the four routes are within 0.2% of each other here
... and reassociation is acceptable	`fastmath` / `-ffast-math`	2.42× on top	results change ($4.6 \cdot 10^{-5}$ here)
array operation a library already has	NumPy / vendor BLAS	up to 1495.9×	only where the operation exists; 4.7× when it does not fit
branch-heavy scan over flat data	Cython or Numba	12×	C and Rust are only 8–17% further ahead
allocation-heavy object graph	an allocator that is not CPython's: C or Rust behind an FFI, or Codon	17.2–20.8× vs 3.9–4.9×	memory-ownership rewrite, or a recompile if Codon takes the source
irregular graph traversal	compile it; a flat layout alone is not the win	9.9× (Cython and C within half a per cent)	flattening it in pure Python was worth 0.89×
object/attribute-heavy Python	upgrade to 3.11+	2.0–3.0× on attribute and method operations	3.11 delivered most of it; 3.12 is slower than 3.11 on 14 of 16 operations; a dict literal on 3.14 runs at 0.74× its 3.10 rate
thread-parallel CPU-bound Python	free-threaded build	5.6–6.1× on 8 threads	1.05–1.14× single-thread tax, paid by the build not the lock
thread-parallel work already in C	threads on any build	5.8–7.2× on 8 threads, balanced kernel	the GIL is irrelevant here; the imbalanced kernel reaches 1.8–3.1×
interpreter build tuning	PGO with full LTO, not PGO alone	1.10× (1.12× adding `-march=native`)	PGO alone is 1.02×; attribute writes lose in all six, 0.88–0.98×
CPython's own tier-2 JIT	measure it: it helps more operations than it hurts	1.04× overall	6 of 16 operations slower (integer arithmetic and attribute writes 0.88–0.89×); still labelled experimental
pure Python that must stay pure Python	PyPy	3.2–42.8×	a `ctypes` call passing a 2 MB buffer costs 5.3× more than CPython's; no Numba; ecosystem support is thinning
a reduction bound by its dependency chain	regroup the accumulation in the source	2.74× at unchanged flags, above `-ffast-math`'s 2.42×	a different summation order, whether a flag or the source grants it (§4.1)
typed hot code, willing to rewrite	CinderX Static Python + JIT	11.4× on top of the JIT	boot sequence documented only in a tutorial; 10.2× slower without the JIT, 5.8× slower even with the specialising interpreter path
startup dominated by imports	Cinder lazy imports	19.7 ms → under 1 μs	fork-only; defers import errors to first use; first touch costs 0.35 ms
mixed pipeline	attack the largest stage, and stop there	1.35× from one stage	Numba on a 3.6% stage gave 1.01×; adding threads gave 0.72× (GIL) and 1.28× (free-threaded)